

TODO University

TODO Department

FPGA Implementation and Verification of Serial and Parallel MDF FFT Architectures

Diploma / Master's Thesis

Author: TODO Author Name

Supervisor: TODO Supervisor

Defense date: TODO Month Year

TODO Month Year

Abstract

The Fast Fourier Transform (FFT) is one of the most widely used kernels in digital signal processing, communications, and scientific computing. When deployed on field-programmable gate arrays (FPGAs), an FFT can be realised in many different shapes that trade silicon area for throughput, latency, and energy. This thesis presents the design, FPGA implementation, and verification of two complementary 1024-point, 16-bit fixed-point single-precision complex FFT cores targeted at the Xilinx Artix-7 xc7a100tcsg324-1 device.

The first design is a compact *serial* FFT processor built around a single radix-2 decimation-in-time butterfly unit, a ping-pong block-RAM data memory, a 1/8-cycle compressed twiddle ROM, and a dynamic block floating-point (BFP) rescaling pipeline. The second design is a *parallel multi-path delay feedback* (MDF) processor with four data paths, eight feedback stages, two no-feedback stages, distributed LUT-RAM delay lines, a commuted complex multiplier that breaks the feedback timing loop, and a ping-pong bit-reversal output buffer. Both cores expose AMBA AXI4-Stream slave/master interfaces and emit a per-block BFP exponent on the TUSER channel.

A unified UVM 1.2 verification environment exercises both architectures with five canonical stimulus families (impulse, DC, sine, multi-tone, chirp) and compares the device-under-test outputs against a NumPy double-precision reference using a signal-to-quantization-noise (SQNR) scoreboard, AXI-Stream SystemVerilog assertions, and a serial-vs-parallel equivalence check.

The two cores were placed and routed in Vivado at 100 MHz. The serial core consumes a small fraction of the Artix-7 fabric (a few thousand LUTs and flip-flops, four DSP slices, eleven block RAMs) and dissipates well under 200 mW, but takes more than N clock cycles to produce a 1024-point block. The parallel MDF core is roughly an order of magnitude larger and dissipates a few hundred milliwatts more, but achieves throughput of one input sample per clock cycle on four parallel lanes, delivering an effective sustained sample rate of 400 MS s^{-1} .

The thesis discusses the mathematical, arithmetic, and architectural choices behind both designs, presents new academic-quality figures for every block and finite-state machine, reports complete simulation, synthesis, place-and-route, timing, power, UVM regression, and functional-coverage results, and a head-to-head trade-off analysis that

quantifies the cost of parallelism in terms of throughput per LUT and throughput per watt. A closing parametric design-space study sweeps both cores over transform size, data width, and twiddle width through a shared figure-of-merit pipeline (SQNR, SFDR, SINAD, ENOB, and phase error), and distils the outcome into transferable design rules — chiefly that accuracy is set by the smaller of the data and twiddle widths at roughly 6 dB per bit.

Keywords: FFT, FPGA, Xilinx Artix-7, radix-2, multi-path delay feedback, MDF, block floating point, AXI4-Stream, UVM, SystemVerilog, fixed-point DSP, SQNR.

Π

Ο Ταχύς Μετασχηματισμός Φουριέ (Fast Fourier Transform, FFT) αποτελεί έναν από τους πιο διαδεδομένους πυρήνες στην ψηφιακή επεξεργασία σήματος, τις επικοινωνίες και τον επιστημονικό υπολογισμό. Όταν υλοποιείται σε αναδιατασσόμενα ολοκληρωμένα κυκλώματα (FPGA), ο FFT μπορεί να πάρει πολλές διαφορετικές μορφές που εξισορροπούν την έκταση πυριτίου, την ρυθμοαπόδοση, την καθυστέρηση και την κατανάλωση ενέργειας. Η παρούσα διπλωματική εργασία παρουσιάζει τη σχεδίαση, την υλοποίηση σε FPGA και την επαλήθευση δύο συμπληρωματικών πυρήνων FFT 1024 σημείων, σταθερής υποδιαστολής 16 bit, στόχου Xilinx Artix-7 xc7a100tcs324-1.

Ο πρώτος σχεδιασμός είναι ένας *σειριακός* επεξεργαστής FFT βασισμένος σε μία μονάδα πεταλούδας radix-2 αποσύνθεσης στον χρόνο, σε δύο τράπεζες block-RAM ping-pong, σε ROM συντελεστών στροφής συμπιεσμένη ανά $1/8$ περιόδου, και σε αγωγό δυναμικής αναπροσαρμογής κλίμακας μπλοκ κινητής υποδιαστολής (Block Floating Point, BFP). Ο δεύτερος σχεδιασμός είναι ένας *παράλληλος* επεξεργαστής Multi-path Delay Feedback (MDF) με τέσσερις παράλληλους δρόμους δεδομένων, οκτώ στάδια ανάδρασης, δύο στάδια χωρίς ανάδραση, γραμμές καθυστέρησης υλοποιημένες σε κατανεμημένη LUT-RAM, μετατοπισμένο μιγαδικό πολλαπλασιαστή που σπάει τον βρόχο χρονισμού της ανάδρασης, και μνήμη αναστροφής bit στην έξοδο. Και οι δύο πυρήνες εκθέτουν διεπαφές AMBA AXI4-Stream και μεταδίδουν τον εκθέτη BFP ανά μπλοκ μέσω του καναλιού TUSER.

Ένα ενοποιημένο περιβάλλον επαλήθευσης UVM 1.2 ασκεί και τις δύο αρχιτεκτονικές με πέντε κανονικές οικογένειες διεγέρσεων (κρουστική, DC, ημιτονική, πολυτονική, chirp) και συγκρίνει τις εξόδους των υπό δοκιμή σχεδιάσεων με αναφορά διπλής ακρίβειας από NumPy, χρησιμοποιώντας βαθμολογητή λόγου σήματος προς θόρυβο κβάντισης (SQNR), ισχυρισμούς SystemVerilog για το πρωτόκολλο AXI-Stream και έλεγχο ισοδυναμίας σειριακού---παράλληλου.

Και οι δύο πυρήνες τοποθετήθηκαν και δρομολογήθηκαν στο Vivado σε 100 MHz. Ο σειριακός πυρήνας καταναλώνει ένα μικρό κλάσμα του υφάσματος του Artix-7 και απορροφά πολύ κάτω από 200 mW, αλλά απαιτεί περισσότερους από N κύκλους ρολογιού για να παράγει ένα μπλοκ 1024 σημείων. Ο παράλληλος πυρήνας MDF είναι περίπου μία τάξη μεγέθους μεγαλύτερος και απορροφά μερικές εκατοντάδες milliwatt περισσότερα, αλλά επιτυγχάνει ρυθμοαπόδοση ενός δείγματος ανά κύκλο σε τέσσερις παράλληλους δρόμους, αποδίδοντας πραγματικό ρυθμό 400 MS^{-1} .

Η εργασία συζητά τις μαθηματικές, αριθμητικές και αρχιτεκτονικές επιλογές πίσω από

τα δύο σχέδια, παρουσιάζει νέα ακαδημαϊκής ποιότητας σχήματα για κάθε μπλοκ και μηχανή πεπερασμένων καταστάσεων, αναφέρει πλήρη αποτελέσματα προσομοίωσης, σύνθεσης, τοποθέτησης και δρομολόγησης, χρονισμού, ισχύος, παλινδρομικών τεστ UVM και λειτουργικής κάλυψης, και μια συγκριτική ανάλυση που ποσοτικοποιεί το κόστος της παραλληλίας σε όρους ρυθμοαπόδοσης ανά LUT και ρυθμοαπόδοσης ανά watt. Μια τελική παραμετρική μελέτη του χώρου σχεδίασης σαρώνει και τους δύο πυρήνες ως προς το μέγεθος μετασχηματισμού, το εύρος δεδομένων και το εύρος των συντελεστών στροφής μέσω ενός κοινού αγωγού δεικτών αξιολόγησης (SQNR, SFDR, SINAD, ENOB και σφάλμα φάσης), και καταλήγει σε μεταφέρσιμους κανόνες σχεδίασης --- κυρίως ότι η ακρίβεια καθορίζεται από το μικρότερο μεταξύ του εύρους δεδομένων και του εύρους των συντελεστών στροφής, με ρυθμό περίπου 6 dB ανά bit.

Λέξεις-κλειδιά: FFT, FPGA, Xilinx Artix-7, radix-2, multi-path delay feedback, MDF, block floating point, AXI4-Stream, UVM, SystemVerilog, αριθμητική σταθερής υποδιαστολής, SQNR.

Acknowledgments

I would like to thank my supervisor, TODO Supervisor, for the guidance, patience, and technical insight provided throughout the course of this thesis. I am also grateful to the members of the laboratory for the discussions on FPGA design, fixed-point arithmetic, and SystemVerilog verification that shaped many of the choices documented in the following chapters.

Finally, I would like to express my gratitude to my family and friends for their continuous support during the years of my studies.

Contents

Abstract	i
II	iii
Acknowledgments	v
List of Acronyms	xiv
1 Introduction	1
1.1 Motivation	1
1.2 Problem Statement	1
1.3 Objectives and Contributions	2
1.4 Thesis Structure	3
2 Mathematical Background of the FFT	5
2.1 The Discrete Fourier Transform	5
2.2 The Cooley–Tukey Decomposition	6
2.3 Twiddle-Factor Symmetries	7
2.4 Fixed-Point Arithmetic	9
2.4.1 Signal-to-Quantization-Noise Ratio	9
2.5 Block Floating Point	10
2.6 Bit-Reversal Output Ordering	11
2.7 Summary	11
3 FPGA Implementation Concepts	12
3.1 The FPGA Fabric	12
3.2 The Xilinx Artix-7 Target Device	13
3.3 Memory Styles	14
3.4 Pipelining	14
3.5 The AXI4-Stream Protocol	15
3.6 The Vivado Design Flow	16
3.7 SAIF-Based Power Analysis	16

3.8	Summary	17
4	Verification Methodology Background	18
4.1	SystemVerilog in a Nutshell	18
4.2	The UVM Class Hierarchy	19
4.3	Functional Coverage and Assertions	20
4.4	Golden-Reference Comparison	20
4.5	Summary	21
5	Serial FFT Architecture	22
5.1	Top-Level Architecture	22
5.2	Top-Level Control FSM	23
5.3	Address Generation Unit	25
5.4	Butterfly Unit	26
5.5	Complex Multiplier (Karatsuba)	26
5.6	Twiddle ROM	27
5.7	Ping-Pong Data Memory	27
5.8	BFP Scanner and Shifter	28
5.9	AXI-Stream Wrapper	28
5.10	Summary	29
6	Parallel MDF FFT Architecture	30
6.1	From SDF to MDF	30
6.2	Top-Level Structure	31
6.3	Feedback Stage Internals	32
6.4	Commuted Twiddle Multiplier	33
6.5	Delay Line	34
6.6	Twiddle ROMs	34
6.7	Block Scaler	34
6.8	No-Feedback Stages	35
6.9	Output Bit-Reversal Buffer	35
6.10	AXI-Stream Wrapper	36
6.11	Summary	36
7	Verification Environment	38
7.1	Unified Environment	39
7.2	AXI-Stream Agent	40
7.3	Stimulus Sequences	40
7.4	Golden Reference Model	41
7.5	Scoreboard and SQNR Computation	41

7.6	Functional Coverage	42
7.7	SystemVerilog Assertions	42
7.8	Equivalence Check	43
7.9	Regression Workflow	43
7.10	Summary	44
8	Results and Comparison	45
8.1	Experimental Setup	45
8.2	Per-Stimulus SQNR	45
8.3	Spectral Figures of Merit	46
8.4	Cross-Architecture Co-Simulation	47
8.5	Coverage and Equivalence	49
8.6	Area Utilisation	50
8.7	Timing Closure	51
8.8	Power	52
8.9	Throughput and Latency	53
8.10	Normalised Efficiency	53
8.11	Discussion	54
9	Parametric Design-Space Study	56
9.1	Sweep Methodology	56
9.2	Structural Parametrizability	56
9.3	Twiddle Width: the Master Precision Knob	57
9.4	Data Width and the min(data, twiddle) Law	57
9.5	Transform Size: Throughput, Latency, and Noise Floor	58
9.6	Failure Mode of the Fixed Exponent	59
9.7	Summary	60
10	Conclusions and Future Work	61
10.1	Summary of Achievements	61
10.2	Limitations	62
10.3	Future Work	63
10.4	Closing Remarks	63
	References	65
A	Selected Code Listings	67
A.1	Address Generation — Count-and-Rotate	67
A.2	Commutated Twiddle Multiplier — LOAD Path	67
A.3	UVM Sine Sequence	68

List of Figures

2.1	Radix-2 decimation-in-time (DIT) butterfly. The twiddle factor W_N^k multiplies the lower input <i>before</i> the addition.	6
2.2	Radix-2 decimation-in-frequency (DIF) butterfly. The twiddle factor multiplies the difference <i>after</i> the addition.	7
2.3	Radix-2 decimation-in-time signal-flow graph for $N = 8$. Solid lines are direct (unity) paths and grey lines are the crossed butterfly paths; the twiddle factor W_8^k labels the lower input of each butterfly. Input is in bit-reversed order, output in natural order.	8
3.1	AXI4-Stream handshake with mid-packet backpressure. The slave de-asserts TREADY during cycles 3–4; the master holds TVALID and TDATA stable for the duration of the stall, and the transfer resumes when TREADY is reasserted.	15
5.1	Top-level block diagram of the serial FFT processor. The AXI-Stream wrapper accepts $N = 1024$ input samples into one of two BRAM banks (Bank 0 / Bank 1), then activates the AGU which schedules $\log_2 N$ stages of butterfly operations. The single butterfly unit (BFU) consumes two operands from the read bank and a twiddle from the ROM and writes the two results back into the write bank, with bank roles swapping every stage. Block floating-point rescaling is applied between stages; the final exponent is reported on the TUSER channel of the AXI master. . .	22
5.2	Top-level control FSM of the serial FFT processor. The machine reverts to Idle after the final output beat is accepted (TLAST & TREADY on M_AXIS). . .	24
6.1	Top-level pipeline of the parallel MDF FFT. Stages 0–7 are feedback (FB) stages whose internal organisation is detailed in Fig. 6.2. Stages 8 and 9 are no-feedback (NF) stages that exploit the trivial twiddles $W_4^0 = 1$, $W_4^1 = -j$, $W_2^0 = 1$ and therefore require no ROM and no complex multiplier. A ping-pong bit-reversal buffer reorders the four parallel output streams into natural bin order before the AXI master.	31

6.2	Internal datapath of a single FB stage in the commuted MDF arrangement. The complex multiplier is taken out of the feedback loop and pipelined three cycles deep on the LOAD path; the BUTTERFLY-phase mux selects <code>sum_d3</code> (the sum output, delayed three cycles to match the multiplier latency) while the LOAD-phase mux selects the multiplier output. The complex multiplier therefore never appears between two consecutive delay-line accesses, and the loop closes comfortably at 100 MHz.	32
7.1	UVM component topology for the unified FFT environment. The active AXI-Stream agent drives the slave port of the DUT; the passive agent observes the master port. Both monitors broadcast on analysis ports consumed by the scoreboard, the coverage collector, and a per-test actuals dumper.	39
8.1	Per-stimulus SQNR comparison. The dashed line marks the ideal $\text{SQNR}_{\text{ideal}} = 6.02B + 1.76 - 10 \log_{10}(N/2) \approx 65$ dB floor for $B=15$, $N=1024$ predicted by Eq. (2.12).	47
8.2	Magnitude-spectrum overlay — Impulse. The two DUT spectra coincide with the reference to within plotting resolution.	48
8.3	Magnitude-spectrum overlay — Sine at bin 50.	49
8.4	Magnitude-spectrum overlay — Multi-tone (bins {37, 113, 271}).	49
8.5	Magnitude-spectrum overlay — Chirp from bin 10 to bin 500. The chirp's broad spectral occupancy is the worst-case stimulus for the parallel core's fixed-shift scaler.	50
8.6	Resource utilisation after Vivado place-and-route.	52
8.7	On-chip power, dynamic plus static.	53
8.8	First-output latency (log scale) and steady-state throughput (linear scale) of the two cores at 100 MHz.	54
8.9	Normalised efficiency. Higher is better in both panels.	55
9.1	SQNR versus data width at 16- and 32-bit twiddles for both cores. At 16-bit twiddles the curves flatten at the twiddle-limited ceiling; at 32-bit twiddles they climb at ≈ 6 dB per bit with no ceiling, illustrating the min(data, twiddle) law of Eq. (9.1).	58
9.2	SQNR versus each swept knob for both cores: twiddle width (left), data width (centre), and transform size N (right). Twiddle and data width climb at ≈ 6 dB per bit until the other dimension limits them; precision versus N is flat for the parallel coherent tone and gently declining for the serial core.	60

List of Tables

3.1	Selected fabric resources of the Xilinx Artix-7 <code>xc7a100tcsg324-1</code> device (data from [10]).	13
3.2	Vivado design flow phases relevant to the present work.	16
7.1	Verification source tree under <code>UVM/</code>	38
7.2	The five canonical stimulus families used by the UVM regression. The amplitude $A_{\text{def}} = 10000$ is the default signed integer amplitude in $Q1.15$ counts. Frequencies f_k are expressed in DFT bin index units, i.e. a sample $x[n] = A \cos(2\pi kn/N)$ has a peak at bin k	40
7.3	SQNR pass/fail thresholds applied by the scoreboard.	42
8.1	Experimental setup.	46
8.2	Per-stimulus SQNR [dB] measured by the scoreboard against a NumPy double-precision reference. Both DUTs pass all five tests.	46
8.3	Single-tone (bin 50) spectral figures of merit from the shared metric pipeline, plus broadband phase error. Entries marked “>120 (cap)” saturate the metric’s 120 dB numerical ceiling and are not literal converter figures — see the discussion below.	48
8.4	Functional coverage achieved by the five-stimulus regression on each DUT.	50
8.5	Serial-vs-parallel offline equivalence (SQNR of one against the other). Threshold: 40 dB.	51
8.6	Vivado placed utilisation on <code>xc7a200tfbg676-2</code> (speed grade <code>-2</code>). Numbers are extracted from <code>*_utilization_placed.rpt</code>	51
8.7	Timing summary after routing (<code>xc7a200tfbg676-2</code> , target 100 MHz, 10 ns period).	52
8.8	On-chip power (Vivado, SAIF-driven post-route estimate).	52
8.9	Throughput and latency at 100 MHz.	53
9.1	Sine-stimulus SQNR versus twiddle width ($N = 1024$, 16-bit data path). Roughly 6.5 dB is gained per twiddle bit on both cores.	57

9.2	Effect of transform size N at 16-bit data and twiddle width. Serial throughput falls with N ; parallel throughput is size-invariant. The parallel impulse underflows at $N = 4096$ (Sec. 9.6).	59
-----	--	----

List of Acronyms

Acronym	Meaning
AGU	Address Generation Unit
AMBA	Advanced Microcontroller Bus Architecture
AXI	Advanced eXtensible Interface
BFP	Block Floating Point
BFU	Butterfly Unit
BRAM	Block Random-Access Memory
CLB	Configurable Logic Block
CDC	Clock-Domain Crossing
DFT	Discrete Fourier Transform
DIF	Decimation In Frequency
DIT	Decimation In Time
DSP	Digital Signal Processing / Digital Signal Processor
DUT	Design Under Test
FF	Flip-Flop
FFT	Fast Fourier Transform
FPGA	Field-Programmable Gate Array
FSM	Finite-State Machine
HDL	Hardware Description Language
IP	Intellectual Property
LUT	Look-Up Table
LUTRAM	Distributed RAM mapped into LUTs
MDF	Multi-path Delay Feedback
NF	No-Feedback (stage)
P&R	Place and Route
PNR	Place and Route
ROM	Read-Only Memory
RTL	Register-Transfer Level
SAIF	Switching Activity Interchange Format
SDF	Single-path Delay Feedback

Chapter 1

Introduction

1.1 Motivation

The Discrete Fourier Transform (DFT) is the single most widely used linear transform in digital signal processing. It underlies the modulation and demodulation of every orthogonal-frequency-division multiplexing (OFDM) waveform in modern wireless standards (Wi-Fi, LTE, 5G NR, DVB-T), the matched filtering in pulse-compression radar, the spectrogram analysis at the heart of every audio codec, and the spectral estimation that drives almost every modern condition-monitoring and vibration-analysis system. Direct evaluation of the DFT has complexity $\mathcal{O}(N^2)$ and is prohibitive even for the modest block sizes routine in these applications; the Fast Fourier Transform (FFT) of Cooley and Tukey [1] reduces the complexity to $\mathcal{O}(N \log N)$ and is what makes the DFT a practical engineering primitive.

When the FFT must be evaluated repeatedly at hard real-time rates — once per OFDM symbol, once per radar pulse, once per audio frame — a software implementation on a general-purpose processor or even on a multi-core DSP is often inadequate. The Field-Programmable Gate Array (FPGA) is the typical implementation platform of choice for such workloads: it offers deterministic latency, the ability to instantiate arbitrary degrees of parallelism, full pipelining at the bit-level, and power-efficient dedicated arithmetic primitives (the DSP48 family on Xilinx 7 Series devices [2]). Open-source HDL flows have made FPGA-based DSP design accessible to a much wider audience than was once the case; closed-source flows (Vivado, Quartus) continue to deliver the timing-closure performance and tooling depth required in production.

1.2 Problem Statement

The FFT admits a wide spectrum of FPGA-friendly architectures that trade silicon area, throughput, latency, energy, and numerical fidelity in different proportions. At

one extreme, a *serial* processor reuses a single butterfly unit across all $\log_2 N$ stages and across all $N/2$ butterflies of each stage, computing one block in roughly N clock cycles with a footprint that fits in a few thousand LUTs and a handful of DSP slices. At the other extreme, a *fully parallel* processor materialises every one of the $\frac{N}{2} \log_2 N$ butterflies in silicon, producing a new output every clock cycle at a price that scales linearly with $N \log N$.

Between these two extremes lies the family of *pipelined* architectures, of which the single-path delay feedback (SDF) and multi-path delay feedback (MDF) variants of He and Torkelson [3, 4] are the most widely deployed. These architectures process one (or P for MDF) samples per clock cycle in a pipeline of $\log_2 N$ butterfly stages, each followed by a delay line whose depth halves with each stage. They occupy an intermediate band of the area-throughput trade-space, with the MDF variant tuneable through the parallelism factor P .

The objective of this thesis is to design, implement, verify, and compare two FPGA FFT processors from opposite sides of this trade-space, applying the same arithmetic conventions, the same target device, the same target clock frequency, and the same verification methodology to both, so that the comparison is direct and the conclusions transferable.

1.3 Objectives and Contributions

The thesis pursues the following four objectives:

1. **Implement a compact serial FFT processor** for $N = 1024$, $Q1.15$ data, with an octant-compressed twiddle ROM, a single Karatsuba complex multiplier, a ping-pong block-RAM data memory, and a dynamic block floating-point rescaling scheme. Wrap the bare core behind an AMBA AXI4-Stream slave/master interface that reports the block exponent on the TUSER sideband. (Chapter 5.)
2. **Implement a parallel multi-path delay feedback (MDF) FFT processor** for $N = 1024$, $P = 4$, $Q1.15$ data, with eight feedback stages, two no-feedback final stages, distributed-LUT-RAM delay lines, a ping-pong bit-reversal output buffer, and a *commuted* complex multiplier that takes the multiplier out of the feedback loop to close timing at 100 MHz. (Chapter 6.)
3. **Verify both cores under a unified SystemVerilog/UVM 1.2 environment** with five canonical stimulus families, an SQNR-based scoreboard against a NumPy double-precision reference, AXI4-Stream concurrent assertions, functional coverage groups, and an offline serial-vs-parallel equivalence check. (Chapters 4–7.)

4. **Compare the two architectures** on a common Xilinx Artix-7 target device under Vivado 2023.2 place-and-route, reporting per-stimulus SQNR, resource utilisation (LUTs, flip-flops, DSP48E1, BRAM), timing slack, SAIF-driven on-chip power, throughput, latency, and the normalised efficiency metrics MS/s-per-LUT and MS/s-per-watt. (Chapter 8.)

The principal contributions of the thesis can be summarised as:

- A self-contained, end-to-end open RTL implementation of both architectures targeted at Xilinx Artix-7, with the same arithmetic conventions, the same AXI-Stream wrapper style, and the same control-flow philosophy on both cores, so that the resulting comparison is methodologically clean.
- A unified parameterised UVM verification environment that treats the serial ($P = 1$) and parallel ($P = 4$) DUTs as instances of the same agent–scoreboard topology, reusing every component except the bus width.
- A quantitative trade-off study — correctness, area, timing, power, throughput, latency, and normalised efficiency — that pins numbers onto the qualitative arguments commonly made about serial vs pipelined FFT architectures.

1.4 Thesis Structure

The remainder of the thesis is organised as follows. **Chapter 2** develops the mathematical foundations of the DFT and the radix-2 Cooley–Tukey FFT, derives the decimation-in-time and decimation-in-frequency butterflies, presents the twiddle-factor symmetries that allow the coefficient ROM to be compressed to one-eighth of its naive size, and introduces the fixed-point and block floating-point arithmetic used throughout. **Chapter 3** reviews the Xilinx Artix-7 fabric, the AMBA AXI4-Stream protocol, the Vivado design flow, and the SAIF-driven power-estimation methodology against which all results are reported. **Chapter 4** introduces the verification methodology used in this thesis: SystemVerilog, UVM 1.2, functional coverage, SystemVerilog Assertions, and golden-reference comparison.

Chapter 5 presents the serial FFT architecture in detail and **Chapter 6** presents the parallel MDF architecture. **Chapter 7** documents the unified UVM verification environment that exercises both cores. **Chapter 8** consolidates the simulation, synthesis, place-and-route, timing, power, and verification results, and presents the head-to-head comparison. **Chapter 9** broadens the single-point comparison into a parametric design-space study that sweeps both cores over transform size, data width, and twiddle width through the same metric pipeline, and distils the result into a set of transferable design

rules. **Chapter 10** summarises the conclusions and outlines the future work that the thesis enables.

Chapter 2

Mathematical Background of the FFT

This chapter develops the mathematical apparatus that underpins both architectures presented in Chapters 5 and 6. It begins with the definition of the Discrete Fourier Transform (DFT), derives the radix-2 Cooley–Tukey Fast Fourier Transform (FFT), discusses the symmetries of the twiddle factors that allow significant memory compression, introduces the fixed-point arithmetic used throughout the implementations, and concludes with the block floating-point rescaling scheme that is applied to control bit growth between stages.

2.1 The Discrete Fourier Transform

Let $x[n]$ be a finite-length complex sequence of N samples, $n = 0, 1, \dots, N - 1$. The Discrete Fourier Transform of $x[n]$ is the length- N complex sequence $X[k]$ defined by [5]

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j\frac{2\pi}{N}kn}, \quad k = 0, 1, \dots, N - 1. \quad (2.1)$$

Introducing the *twiddle factor*

$$W_N^{kn} = e^{-j\frac{2\pi}{N}kn}, \quad (2.2)$$

the DFT becomes the matrix–vector product

$$X[k] = \sum_{n=0}^{N-1} x[n] W_N^{kn}, \quad (2.3)$$

and the inverse DFT differs only in the sign of the exponent and a $1/N$ scale factor. A direct evaluation of (2.3) requires N complex multiply–accumulate operations per output bin, for a total of N^2 complex multiplications and $N(N - 1)$ complex additions. The quadratic complexity is prohibitive for the block sizes encountered in modern signal

processing (e.g. $N = 1024$ for the OFDM symbols studied in this thesis would require more than 10^6 complex multiplications per block).

The Fast Fourier Transform is any algorithm that exploits the algebraic structure of W_N^{kn} to reduce this complexity. The earliest practical formulation, popularised by Cooley and Tukey [1], requires only $\mathcal{O}(N \log_2 N)$ operations for N a power of two; for $N = 1024$ this is a reduction by a factor of roughly 100 that has shaped almost every later signal processing architecture, including the two designs documented in this thesis [6].

2.2 The Cooley–Tukey Decomposition

The classical radix-2 Cooley–Tukey algorithm assumes $N = 2^p$. The sum (2.3) is split into two half-length sums, one over the even-indexed input samples and one over the odd-indexed input samples,

$$X[k] = \underbrace{\sum_{r=0}^{N/2-1} x[2r] W_N^{2rk}}_{E[k]} + W_N^k \underbrace{\sum_{r=0}^{N/2-1} x[2r+1] W_N^{2rk}}_{O[k]}. \quad (2.4)$$

Because $W_N^2 = W_{N/2}^1$, the sums $E[k]$ and $O[k]$ are each a length- $(N/2)$ DFT of the even- and odd-indexed subsequences, respectively. Substituting back gives the fundamental decimation-in-time (DIT) recursion

$$X[k] = E[k] + W_N^k O[k], \quad (2.5)$$

$$X[k + N/2] = E[k] - W_N^k O[k], \quad (2.6)$$

for $k = 0, 1, \dots, N/2 - 1$. The decomposition is applied recursively to each half-length DFT until the trivial $N = 2$ case is reached. At each level of the recursion, $N/2$ *butterflies* of the form depicted in Fig. 2.1 are evaluated. The total number of complex multiplications is $\frac{N}{2} \log_2 N$ and the total number of complex additions is $N \log_2 N$ [5].

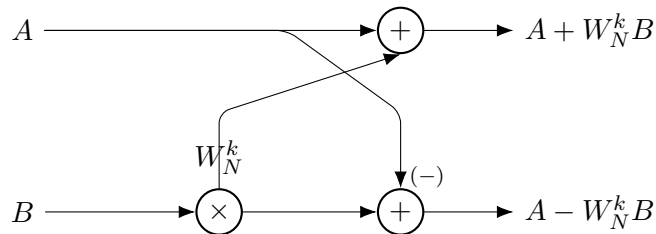


Figure 2.1: Radix-2 decimation-in-time (DIT) butterfly. The twiddle factor W_N^k multiplies the lower input *before* the addition.

A dual derivation that splits the *output* index k instead of the input index n yields

the decimation-in-frequency (DIF) formulation, in which the twiddle multiplication appears *after* the addition:

$$X[2k] = \sum_{n=0}^{N/2-1} (x[n] + x[n + N/2]) W_{N/2}^{nk}, \quad (2.7)$$

$$X[2k + 1] = \sum_{n=0}^{N/2-1} (x[n] - x[n + N/2]) W_N^n W_{N/2}^{nk}. \quad (2.8)$$

The resulting butterfly is shown in Fig. 2.2. DIT and DIF have identical operation counts; they differ only in the *ordering* of the addition and the twiddle multiplication, which in turn determines whether the input sequence must be presented in bit-reversed order (DIT) or whether the output sequence is produced in bit-reversed order (DIF). The serial processor of Chapter 5 uses DIT, while the parallel processor of Chapter 6 uses DIF; the consequences of this choice on memory addressing and output reordering are discussed in the respective architecture chapters.

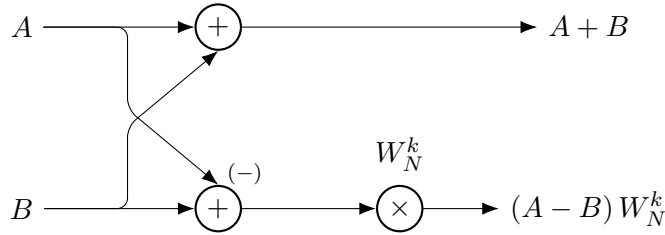


Figure 2.2: Radix-2 decimation-in-frequency (DIF) butterfly. The twiddle factor multiplies the difference *after* the addition.

Cascading the butterflies across all $\log_2 N$ levels of the recursion yields the complete *signal-flow graph* (SFG) of the transform. Fig. 2.3 shows the SFG for the smallest non-trivial case, $N = 8$, in decimation-in-time form. The input is consumed in bit-reversed order $x_0, x_4, x_2, x_6, x_1, x_5, x_3, x_7$ and the output emerges in natural order $X_0, \dots, X_7$. At stage m ($m = 1, 2, 3$) the two nodes of each butterfly are separated by a span of 2^{m-1} samples, and the twiddle factor applied to butterfly t of that stage is $W_8^{(8t)/2^m}$; stage 1 therefore uses only $W_8^0 = 1$, stage 2 adds $W_8^2 = -j$, and stage 3 uses the full set $W_8^0, W_8^1, W_8^2, W_8^3$. The 1024-point graph implemented by the serial processor of Chapter 5 is the identical structure scaled to $\log_2 1024 = 10$ stages and $\frac{N}{2} \log_2 N = 5120$ butterflies.

2.3 Twiddle-Factor Symmetries

A naive implementation of the radix-2 FFT stores the full set of N twiddle factors W_N^k , $k = 0, 1, \dots, N - 1$, in a coefficient ROM. The twiddle factors are samples of the unit circle and exhibit several useful symmetries that allow this storage to be reduced

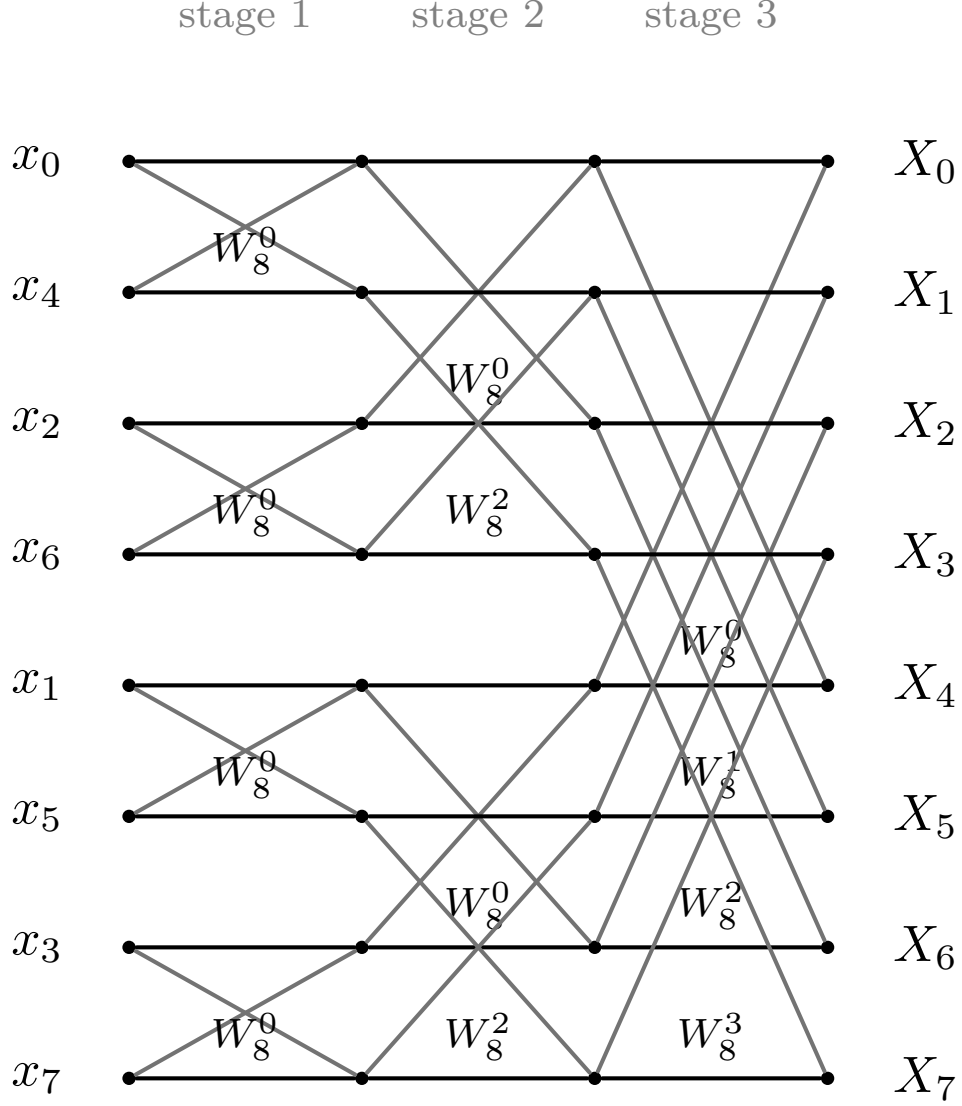


Figure 2.3: Radix-2 decimation-in-time signal-flow graph for $N = 8$. Solid lines are direct (unity) paths and grey lines are the crossed butterfly paths; the twiddle factor W_8^k labels the lower input of each butterfly. Input is in bit-reversed order, output in natural order.

significantly. Three properties are exploited by both designs in this thesis:

$$W_N^{k+N/2} = -W_N^k \quad (\text{half-circle anti-symmetry}) \quad (2.9)$$

$$W_N^{N/4-k} = -j \overline{W_N^k} \quad (\text{quarter-circle reflection}) \quad (2.10)$$

where $\overline{(\cdot)}$ denotes complex conjugation. The combination of (2.9) and (2.10) allows the twiddle ROM to store only the first quadrant ($k = 0, \dots, N/4$). A further application of the cosine-sine relation $\cos(\pi/2 - \theta) = \sin(\theta)$ identifies the cosine table for the second half of the quadrant with the sine table for the first half, reducing the storage to the first *octant*, $k = 0, \dots, N/8$. The octant index and a few sign-flip and real/imaginary-swap bits, derived from the upper bits of k , are sufficient to reconstruct W_N^k for any k .

Both architectures exploit this 1/8-cycle compression, reducing the twiddle ROM from N entries to $N/8 + 1 = 129$ entries for $N = 1024$.

2.4 Fixed-Point Arithmetic

All complex data in both architectures are represented in $Q1.15$ two's-complement fixed-point format. A $Q1.15$ number consists of one sign bit and fifteen fractional bits, representing values in the half-open interval $[-1, 1)$ with a resolution of $2^{-15} \approx 3.05 \times 10^{-5}$. The real and imaginary parts of each sample are stored as independent $Q1.15$ words, packed into a 32-bit AXI beat with the real part in the upper half-word and the imaginary part in the lower half-word.

A complex multiplication of two $Q1.15$ numbers produces a $Q2.30$ intermediate result that must either be truncated or rounded back to $Q1.15$. Truncation is biased and produces a noticeable DC offset in the output, while a simple round-to-nearest introduces no bias but requires an additional adder. Both implementations in this thesis perform a round-half-up by adding the constant 2^{14} before the truncation shift, which is the cheapest implementation that preserves the unbiased mean.

A radix-2 butterfly increases the dynamic range of its output by one bit, because $|A \pm WB|$ may be as large as $|A| + |B|$. Across the $\log_2 N = 10$ stages of a 1024-point FFT this would require a worst-case internal data path width of $16 + 10 = 26$ bits. Two strategies are commonly used to manage this bit growth:

- *Scaling at every stage* by a fixed right shift of one bit. This is the cheapest in area but introduces a worst-case SNR loss of 6 dB per stage on signals that do not actually grow [7]. This is the strategy used by the parallel MDF processor of Chapter 6, where the feedback delay lines must operate at a fixed width and where the per-stage area is the dominant cost.
- *Block floating point* (BFP), where each stage performs a *conditional* right shift driven by the maximum magnitude observed at the previous stage. A single integer exponent is tracked across the entire block and re-applied at the end. This strategy is used by the serial processor of Chapter 5, where one BFP exponent can be amortised across all N butterflies of each stage.

Both strategies are discussed in the next section.

2.4.1 Signal-to-Quantization-Noise Ratio

The fidelity of a fixed-point FFT is conventionally quantified by the *signal-to-quantization-noise ratio* (SQNR), defined as the ratio of the power of the floating-point reference

output to the power of the difference between the floating-point reference and the fixed-point implementation:

$$\text{SQNR} = 10 \log_{10} \left(\frac{\sum_{k=0}^{N-1} |X_{\text{ref}}[k]|^2}{\sum_{k=0}^{N-1} |X_{\text{ref}}[k] - X_{\text{dut}}[k]|^2} \right) \quad [\text{dB}]. \quad (2.11)$$

Throughout this thesis the floating-point reference X_{ref} is the double-precision NumPy `numpy.fft.fft` of the same $Q1.15$ input vector that is presented to the DUT. The expectation under a uniform-noise model is [7]

$$\text{SQNR}_{\text{ideal}} \approx 6.02 B + 1.76 - 10 \log_{10}(N/2) \quad [\text{dB}], \quad (2.12)$$

where B is the number of magnitude bits. For $B = 15$ and $N = 1024$ this predicts $\text{SQNR}_{\text{ideal}} \approx 65$ dB, which is the target floor against which the measured results in Chapter 8 are evaluated.

2.5 Block Floating Point

The fixed-point strategy used by the serial processor is block floating point. The data path width is kept at 16 bits but a single shared integer exponent e , called the *block exponent*, is maintained across all N samples. The mathematical interpretation is that the hardware stores the mantissas of the samples and the host scales them by 2^{-e} when interpreting the output. Conceptually, each butterfly stage performs the following three-step procedure:

1. Compute all $N/2$ butterflies of the stage at full bit width (17 bits per component to absorb the worst-case 1-bit growth).
2. Detect the maximum magnitude across the stage by counting the leading sign bits (*count leading zeros*, CLZ) of every butterfly output.
3. Right-shift the entire stage output by 0 or 1 bit depending on whether bit growth was actually observed and update the block exponent accordingly. The shift is performed by a single barrel shifter shared with the data path.

At the end of all $\log_2 N$ stages, the accumulated block exponent is emitted as an 8-bit value on the TUSER sideband of the AXI output stream, alongside the mantissas of $X[k]$. The host reconstructs the true magnitude by computing $|X_{\text{true}}[k]| = |X_{\text{stored}}[k]| \cdot 2^e$.

Compared to fixed scaling, BFP avoids the cumulative 6 dB loss per stage on low-amplitude signals and recovers close to the $\text{SQNR}_{\text{ideal}}$ floor of (2.12); the cost is one barrel shifter and one CLZ tree per stage. As shown in Chapter 8, the serial processor

achieves an SQNR within a few decibels of (2.12) on every test signal, while the parallel MDF processor with fixed shift-by-1 falls roughly 20 to 30 dB short on the most demanding chirp stimulus.

2.6 Bit-Reversal Output Ordering

A direct in-place implementation of the recursion (2.6) or (2.8) produces results in *bit-reversed* index order. Specifically, if the input is presented in natural order $x[0], x[1], \dots, x[N-1]$ to a DIT FFT, the output appears in bit-reversed order $X[\text{br}(k)]$, where $\text{br}(\cdot)$ denotes reversal of the $\log_2 N$ bits of the index. The dual holds for DIF: natural-order input is fed into an in-place computation whose output emerges in bit-reversed order and must be reordered before consumption.

The two architectures handle the reordering at opposite ends of the pipeline. The serial DIT processor accepts AXI input in natural order into one bank of the data memory and the AGU performs an implicit bit-reverse when reading the input for the first stage; the output emerges in natural order. The parallel DIF processor, in contrast, computes the FFT in place across its delay lines, and a dedicated bit-reversal buffer (Sec. 6.9) reorders the four parallel streams of bit-reversed samples into natural order before they are emitted on the AXI master.

2.7 Summary

The chapter has introduced the DFT, derived the radix-2 Cooley–Tukey FFT in both DIT and DIF forms, identified the twiddle-factor symmetries that reduce coefficient ROM to one-eighth of its naive size, presented the $Q1.15$ fixed-point format and the SQNR figure of merit, and described the block floating-point rescaling scheme. The terminology and symbols established here are used without further definition in Chapters 5, 6, and 8.

Chapter 3

FPGA Implementation Concepts

This chapter establishes the implementation context for the two FFT cores. Section 3.1 reviews the SRAM-based FPGA fabric and Section 3.2 the specific resources of the Xilinx Artix-7 device targeted by this thesis. Section 3.3 compares the memory styles available on the device. Section 3.4 summarises the role of pipelining in timing closure. Section 3.5 describes the AMBA AXI4-Stream protocol through which both cores communicate with the host system. Section 3.6 traces the Vivado design flow from RTL to bitstream, and Section 3.7 explains the switching-activity-based power-estimation methodology used in Chapter 8.

3.1 The FPGA Fabric

A modern field-programmable gate array (FPGA) is a regular array of fine-grained programmable logic cells interconnected by a routable switch matrix [8]. The user maps a register-transfer-level (RTL) description onto this fabric, and the configuration that implements the design is loaded into the device at power-on as a *bitstream*. The most common building block in commercial SRAM-based FPGAs is the *look-up table* (LUT), a small RAM-backed truth-table that can implement any Boolean function of its inputs. In the Xilinx 7 Series family used in this thesis, each LUT has six inputs and can implement any 6-input combinational function or two 5-input functions sharing inputs [9].

LUTs are paired with edge-triggered *flip-flops* (FFs) inside a *slice*, which also contains dedicated carry chains, multiplexers, and shift-register/distributed-RAM primitives. Slices are grouped into *configurable logic blocks* (CLBs); two slices per CLB in the 7 Series family. The CLB array provides the bulk of the combinational and sequential logic of any user design. Dedicated columns of *Block RAM* (BRAM) and *DSP48* primitives are interleaved through the fabric and provide bulk storage and high-speed signed arithmetic, respectively.

Routing between primitives is performed through a programmable switch matrix consisting of segmented wires of various lengths and SRAM-configured pass transistors. Routing delay typically dominates combinational delay in deep designs, which makes pipelining (Sec. 3.4) the single most effective tool for closing timing on complex DSP cores.

3.2 The Xilinx Artix-7 Target Device

Both FFT cores in this thesis target the Xilinx Artix-7 xc7a100tcsg324-1 device, a member of the 7 Series 28 nm family marketed at price-sensitive embedded applications. The relevant fabric resources of this device are summarised in Tab. 3.1.

Table 3.1: Selected fabric resources of the Xilinx Artix-7 xc7a100tcsg324-1 device (data from [10]).

Resource	Count
Logic cells	101 440
Slices	15 850
6-input LUTs	63 400
Flip-flops	126 800
LUTs configurable as distributed RAM	19 000
36 Kb Block RAM tiles	135
DSP48E1 slices	240
Maximum clock frequency, -1 grade	up to 464 MHz

Two dedicated primitives are of particular importance for FFT implementations.

DSP48E1 slice. Each DSP48E1 is a 25×18 -bit two’s-complement signed multiplier feeding a 48-bit pre-adder and a 48-bit post-accumulator, with optional input and output pipeline registers [2]. Inferred multipliers wider than the native 25×18 are mapped to multiple DSP48E1 instances connected via the dedicated cascade ports. In this thesis the Karatsuba complex multiplier (Sec. 5.5) maps to three DSP48E1 slices per real-by-real 16×16 multiplication, all internally pipelined.

Block RAM tile. Each 36 Kb BRAM tile is configurable as a true dual-port memory with independent address, write-enable, and clock per port, supports configurable widths from 1 to 72 bits, and can be split into two independent 18 Kb half-tiles [11]. The serial FFT processor of Chapter 5 uses BRAM to implement the two 1024-word ping-pong data banks; the parallel MDF processor of Chapter 6 uses BRAM only for the small ping-pong bit-reversal buffer at the output, while the per-stage delay lines are inferred into distributed LUT-RAM.

3.3 Memory Styles

Three styles of read/write memory are commonly used in 7 Series designs.

Block RAM (BRAM) is the dedicated 36 Kb dual-port primitive described above.

It is the most area-efficient choice for memories larger than a few hundred words, and it is the only primitive that supports synchronous read with single-cycle latency at hundreds of megahertz. Its drawback is the 1-cycle read latency, which the address generation unit of Sec. 5.3 must explicitly compensate for.

Distributed RAM (LUTRAM) uses the 64-bit SRAM behind each 6-LUT as a small *asynchronous*-read memory. It is appropriate for very small memories (a few hundred bits) and is ideal for the short, exact-depth delay lines used by the feedback stages of the MDF processor (Sec. 6.5). Inference is controlled by the `ram_style = "distributed"` synthesis attribute.

Shift-register LUT (SRL) cascades the same 64-bit SRAM cell into a 32- or 64-deep variable-tap shift register. SRLs are even cheaper than LUTRAM for small constant-depth shift registers (one LUT per 32 bits). The MDF delay lines of depth ≤ 32 are inferred as SRL chains automatically by Vivado.

3.4 Pipelining

In a clocked synchronous design the maximum operating frequency is $f_{\max} = 1/T_{\text{cp}}$, where T_{cp} is the longest combinational delay between any two registers on the critical path, plus the clock-to-Q delay of the source register, the setup time of the destination register, and the worst-case clock skew. Any combinational path that exceeds the budgeted T_{cp} produces *negative slack* at static timing analysis and the design fails to close timing.

The remedy is to *pipeline* long combinational paths by inserting intermediate registers, partitioning the path into stages each of which fits within the clock period. Pipelining trades *latency* (the number of cycles from input to output) for *throughput* (the number of new inputs that can be accepted per cycle) and for *area* (the additional flip-flops). On Xilinx 7 Series fabric, the DSP48E1 itself contains three internal pipeline-register stages (M register, P register, OPMODE register) that can be enabled selectively without consuming additional CLB resources. Both architectures in this thesis exploit these internal registers fully: the serial processor's complex multiplier is implemented as a 4-stage pipeline and the MDF processor's commuted complex multiplier (Sec. 6.4) as a 3-stage pipeline.

3.5 The AXI4-Stream Protocol

Both FFT cores are wrapped behind the AMBA AXI4-Stream protocol [12], the uni-directional streaming subset of the AXI family of bus standards developed by ARM. AXI4-Stream defines a one-way point-to-point channel between a *master* that produces data and a *slave* that consumes it. The minimal set of signals used in this thesis is

- **TVALID** — driven by the master; high when the data on **TDATA** is valid;
- **TREADY** — driven by the slave; high when the slave can accept a beat in the current cycle;
- **TDATA** — payload, 32 bits per beat for the serial core, 128 bits per beat for the parallel core (four complex samples in parallel);
- **TLAST** — end-of-packet marker; high on the last beat of each N -sample block;
- **TUSER** — application-specific sideband used here to emit the block floating-point exponent on the master output of the serial core (8 bits) or the per-stage scaling count on the master output of the MDF core (4 bits).

The handshake rule states that a beat is transferred only on rising clock edges where both **TVALID** and **TREADY** are asserted. Either party may withhold its signal to introduce a wait cycle. Crucially, once **TVALID** is asserted the master is not allowed to deassert it until at least one beat has been accepted; this restriction is verified by the SystemVerilog assertions described in Sec. 7.7. A short timing example is shown in Fig. 3.1, in which the slave stalls for two cycles in the middle of a four-beat packet.

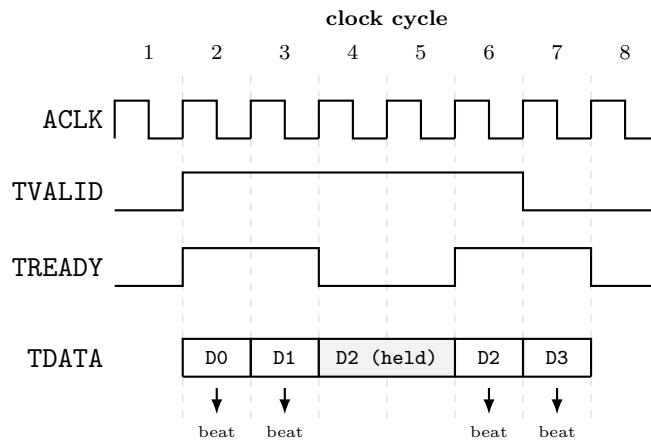


Figure 3.1: AXI4-Stream handshake with mid-packet backpressure. The slave deasserts **TREADY** during cycles 3–4; the master holds **TVALID** and **TDATA** stable for the duration of the stall, and the transfer resumes when **TREADY** is reasserted.

3.6 The Vivado Design Flow

Both architectures were taken from RTL to placed-and-routed implementation with Vivado 2023.2. The flow consists of seven distinct phases, each consuming the artefacts of the previous phase (Tab. 3.2).

Table 3.2: Vivado design flow phases relevant to the present work.

Phase	Tool	Output
Elaboration	<code>synth_design -rtl</code>	RTL netlist
Synthesis	<code>synth_design</code>	Generic netlist (<code>.dcp</code>)
Logic opt.	<code>opt_design</code>	Optimised netlist
Placement	<code>place_design</code>	Placed checkpoint
Phys. opt.	<code>phys_opt_design</code>	Re-timed placed checkpoint
Routing	<code>route_design</code>	Routed checkpoint
Bitstream	<code>write_bitstream</code>	Programming file (<code>.bit</code>)

Constraint information enters the flow via Xilinx Design Constraint (XDC) files. The two XDC files used in this thesis, `constrs/fft_axi_top.xdc` for the serial core and `constraints/fft_axi_top.xdc` for the parallel core, specify a 100 MHz clock with the standard `create_clock` constraint and declare the AXI input/output ports as `set_input_delay` / `set_output_delay` relative to the same clock. Static timing analysis after routing produces the worst negative slack (WNS), total negative slack (TNS), and worst hold slack (WHS) numbers reported in Chapter 8.

3.7 SAIF-Based Power Analysis

Power dissipation on an FPGA has three components: *static* power, driven primarily by transistor leakage and fixed by the device choice; *dynamic* power, proportional to $CV^2f\alpha$ where α is the toggle activity of each net; and *I/O* power, which depends on the chosen I/O standard and output loading. Of these, only the dynamic component is design-dependent and can be optimised by the designer.

A meaningful estimate of dynamic power therefore requires an estimate of the toggle activity of every signal in the design under representative input stimulus. The Switching Activity Interchange Format (SAIF) [13] is a standard ASCII file format in which the toggle counts and high/low durations of every signal are recorded over the duration of a simulation. The flow used in this thesis is:

1. Run a behavioural simulation of the post-synthesis netlist with a representative input stimulus (a sine vector for the serial core, a chirp vector for the parallel core).

2. Dump the activity into a SAIF file using the `write_saif` simulator command.
3. Pass the SAIF file to `report_power` in the routed Vivado checkpoint via the `-input` argument of the Tcl script `scripts/power_with_saif.tcl`.

The resulting power report partitions the total dissipation into static, signal, logic, BRAM, DSP, clock, and I/O contributions. The headline numbers from these reports are tabulated in Sec. 8.8.

3.8 Summary

The Artix-7 fabric provides the LUTs, flip-flops, BRAMs, distributed RAMs, and DSP48E1 multiplier-accumulator slices that the two FFT cores target. Pipelining drives all critical-path decisions; the AXI4-Stream interface drives all timing budgets at the system boundary; and the Vivado flow with SAIF-driven power analysis produces the area, timing, and power numbers consolidated in Chapter 8.

Chapter 4

Verification Methodology Background

The two FFT cores presented in Chapters 5 and 6 were verified with a unified SystemVerilog/UVM testbench described in detail in Chapter 7. The present chapter introduces the methodological vocabulary required to read that chapter: a short overview of SystemVerilog (Sec. 4.1), the standard UVM class hierarchy (Sec. 4.2), functional coverage and SystemVerilog Assertions (Sec. 4.3), and the golden-reference comparison flow used to grade each test run (Sec. 4.4).

4.1 SystemVerilog in a Nutshell

SystemVerilog is the IEEE 1800 standard [14] extension to the original Verilog hardware description language. Beyond the synthesisable RTL subset that overlaps Verilog 2001, it adds a rich set of constructs aimed at *verification*:

- object-oriented classes with single inheritance, virtual methods, parameterised types, and constrained-random class members;
- *interfaces*, which bundle related signals into a single named connection point that can carry its own *clocking blocks* and *modports*;
- *covergroups*, which sample user-defined cross products of signal values and produce a coverage database;
- *concurrent assertions* written in the SystemVerilog Assertion (SVA) language, which describe temporal properties of the design and continuously check them in simulation.

The verification class library at the heart of this thesis — UVM 1.2 [15] — is itself a portable SystemVerilog library that exploits all of these constructs.

4.2 The UVM Class Hierarchy

The Universal Verification Methodology is a base-class library standardised by Accellera and widely adopted across the EDA industry. It mandates a specific class hierarchy whose components communicate through transaction-level interfaces and whose lifecycle is governed by the predefined UVM *phases* (`build_phase`, `connect_phase`, `run_phase`, ...). The class hierarchy used in the testbench of this thesis follows the canonical UVM topology shown later in Fig. 7.1 and consists of:

Transaction (sequence item). A class derived from `uvm_sequence_item` that bundles one logical transaction (here, one AXI4-Stream beat) into a single object carrying its data, sideband fields, and optional constraints.

Sequence. A class derived from `uvm_sequence` that generates a stream of sequence items. The testbench of Chapter 7 defines one sequence per stimulus family (impulse, DC, sine, multi-tone, chirp) plus a regression sequence that runs them all.

Sequencer. An arbitrator that owns the pending sequence items and feeds them to the driver one at a time.

Driver. Converts sequence items into pin-level activity on a virtual interface, respecting the protocol's handshake rules.

Monitor. Observes the same virtual interface and reconstructs sequence items, broadcasting them on an analysis port for the rest of the environment to consume.

Agent. A unit of reuse that bundles a driver, monitor, and sequencer for a single protocol on a single interface. May be *active* (drives the bus) or *passive* (only observes).

Scoreboard. Consumes monitor analysis transactions from both the input and output sides of the device under test, applies a reference model, and reports per-test pass/fail.

Environment. Instantiates and connects the agents, the scoreboard, and the coverage collectors for a particular DUT.

Test. The top-level entry point: instantiates the environment, selects the sequence to start, and exits when the sequence completes.

This separation between protocol-level mechanics (in the agent), DUT correctness (in the scoreboard), and stimulus selection (in the sequence and test) is the central organising principle of UVM and is what allows the same agent to be reused for the serial and parallel FFT cores by re-parameterising the bus width (Sec. 7.2).

4.3 Functional Coverage and Assertions

Two complementary instruments are used in UVM to demonstrate that the verification effort has been thorough.

Functional coverage is measured with `covergroup` constructs that sample user-defined points or crosses at well-chosen events. A coverage point may, for example, bin the amplitude of the input signal into ten ranges; a cross may then multiply that amplitude binning by a five-bin stimulus-kind enum, producing a 50-bin cross-coverage table. The percentage of bins hit by a given regression is a quantitative measure of how exhaustively the stimulus has explored the input space. The four cover groups used in this thesis (signal type, amplitude bucket, BFP exponent, peak bin crossed with signal type) are detailed in Sec. 7.6.

SystemVerilog Assertions (SVA) are concurrent properties written in a temporal logic that targets the rising clock edge. A typical assertion declares that whenever `TVALID` is asserted but `TREADY` is not, `TDATA` must be stable on the next cycle. Such properties are continuously evaluated by the simulator and report immediately when violated. They serve both as protocol checkers (catching DUT violations of the AXI handshake rules of Sec. 3.5) and as observability hooks for *cover properties*, which count how often interesting protocol events actually occur during simulation. The five protocol assertions and three cover properties used in this thesis are listed in Sec. 7.7.

4.4 Golden-Reference Comparison

The scoreboard does not embed a SystemVerilog implementation of the FFT. Instead, the reference output for every stimulus is precomputed offline by the Python script `UVM/scripts/gen_refs.py` using `numpy.fft.fft` at double precision. The script writes the *Q1.15* input vector and the corresponding floating-point reference output to text files in `UVM/refs/`, one pair per stimulus. At elaboration the scoreboard loads the reference file, casts it to double precision, and at run time accumulates the DUT output samples until `TLAST` is observed. The collected DUT vector is then compared against the reference using the SQNR formula of (2.11), and a per-test pass/fail decision is emitted based on a per-stimulus SQNR threshold.

The choice to precompute the reference — as opposed to invoking a DPI-C call to a live FFT library — has three practical advantages: the testbench is bit-exact reproducible across simulators, no foreign language interface is required (which makes it portable to `xsim`), and the reference vector can be inspected and visualised independently of any simulation run.

4.5 Summary

UVM provides the agent / scoreboard / environment / test scaffolding, SVA provides the protocol-level safety net, and a NumPy-driven golden reference provides the absolute correctness criterion. With this vocabulary in place, Chapter 7 can describe the specific testbench that exercises the two FFT cores.

Chapter 5

Serial FFT Architecture

This chapter presents the first of the two designs: a compact serial processor that computes a 1024-point complex FFT using a *single* radix-2 decimation-in-time butterfly unit shared across all $\log_2 N = 10$ stages and all $N/2 = 512$ butterflies per stage. The processor fits in a small fraction of an Artix-7 `xc7a100t` device and dissipates well under 200 mW; it pays for these properties in latency, taking roughly N clock cycles to process a single block. Section 5.1 introduces the top-level architecture, Section 5.2 the control finite-state machine that schedules its operation, and Sections 5.3–5.9 describe each datapath submodule in turn.

5.1 Top-Level Architecture

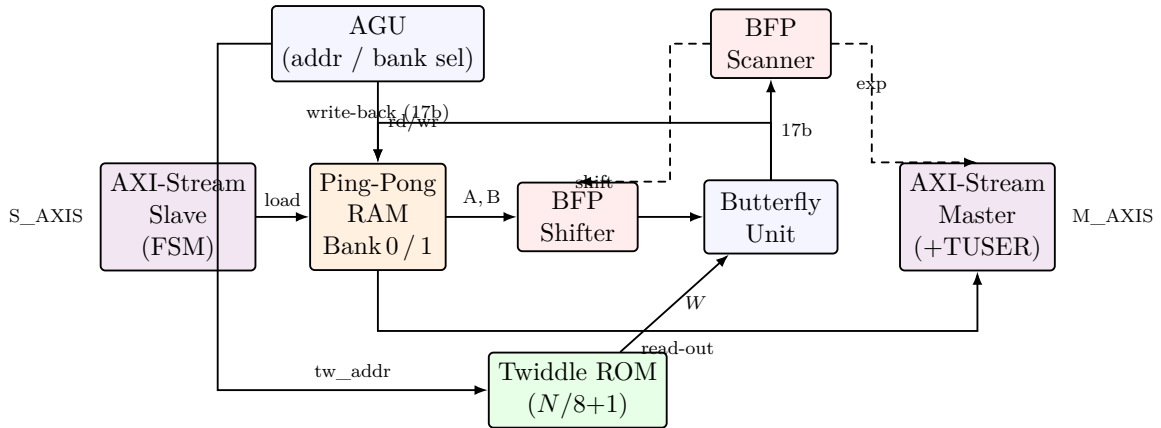


Figure 5.1: Top-level block diagram of the serial FFT processor. The AXI-Stream wrapper accepts $N = 1024$ input samples into one of two BRAM banks (Bank 0 / Bank 1), then activates the AGU which schedules $\log_2 N$ stages of butterfly operations. The single butterfly unit (BFU) consumes two operands from the read bank and a twiddle from the ROM and writes the two results back into the write bank, with bank roles swapping every stage. Block floating-point rescaling is applied between stages; the final exponent is reported on the **TUSER** channel of the AXI master.

The block diagram of Fig. 5.1 shows the seven sub-blocks of the serial processor:

1. the **AXI-Stream wrapper** `fft_axi_top`, which owns the top-level FSM and bridges the AXI handshake to the bare-core preload, compute, and readout interfaces;
2. the **address generation unit** (AGU), which generates the read and delayed write addresses, bank selectors, and twiddle-ROM indices for every butterfly of every stage;
3. the **ping-pong data memory**, two true dual-port BRAM banks (Bank 0 and Bank 1) of $1024 \text{ words} \times 17 + 17 = 34$ bits each;
4. the **twiddle ROM** with octant compression, storing $N/8 + 1 = 129$ complex coefficients;
5. the **butterfly unit** with its embedded 4-stage-pipelined Karatsuba complex multiplier;
6. the **BFP scanner**, a count-leading-sign tree that examines all butterfly outputs of a stage and decides whether to right-shift the next stage's inputs;
7. the **BFP shifter**, a barrel shifter that performs the per-stage conditional right shift.

All sub-blocks operate on a single clock domain with a single synchronous active-high reset.

A complete block-floating-point exponent is accumulated across the ten stages. When the AGU asserts `fft_done`, the AXI wrapper latches the exponent into an 8-bit register and emits it on `m_axis_tuser` stable across all N output beats; the host reconstructs the true magnitude by multiplying each $Q1.15$ output by 2^{TUSER} .

5.2 Top-Level Control FSM

The AXI-Stream wrapper is structured around a four-state finite-state machine that orchestrates a single block. The state diagram is shown in Fig. 5.2.

Idle. The default state on reset and after every completed block. `S_AXIS_TREADY` is asserted (provided the master output register is empty) so that the master may begin a new block at any time. The FSM advances to Load on the first asserted `S_AXIS_TVALID`.

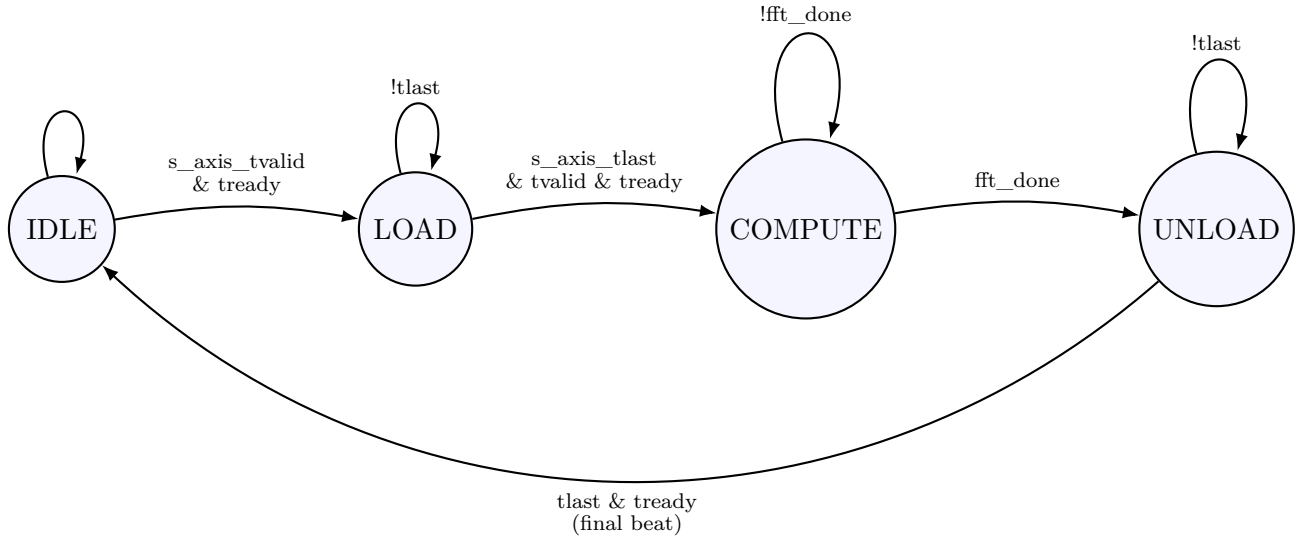


Figure 5.2: Top-level control FSM of the serial FFT processor. The machine reverts to Idle after the final output beat is accepted (TLAST & TREADY on M_AXIS).

Load. The FSM keeps `S_AXIS_TREADY` asserted while N samples are written into Bank 0 of the data memory. The wrapper performs an *in-hardware bit-reversal* of the load counter so that the samples are stored at addresses $\text{br}(n)$ rather than at sequential addresses. This is required because the DIT butterfly expects its inputs in bit-reversed order: the equivalent of the explicit `bit_reverse_permutation` step that the Python reference flow performs before `$readmemh` is folded directly into the address generator at synthesis time. The transition to Compute occurs the same cycle that `S_AXIS_TLAST` is observed.

Compute. The wrapper asserts the `start_fft` pulse to the AGU and deasserts `S_AXIS_TREADY`. The AGU runs through all $\log_2 N = 10$ stages, $N/2$ butterflies each, writing the intermediate results back to the alternate bank between stages. Because $\log_2 N$ is even, after ten stages the result is in Bank 0 again; for odd $\log_2 N$ it would be in Bank 1. The transition to Unload occurs the cycle the AGU asserts `fft_done`, at which point the wrapper latches the final block exponent and pre-issues the first BRAM read address.

Unload. The wrapper walks the result bank sequentially from address 0 to $N - 1$, presenting each bin on `M_AXIS_TDATA` with the latched exponent on `M_AXIS_TUSER`. `M_AXIS_TLAST` is asserted on the final beat. The state machine respects the AXI rule that, once `M_AXIS_TVALID` has been asserted, it must remain asserted with stable data until `M_AXIS_TREADY` is observed; this requires three internal phases inside the Unload state (data not yet ready, present-and-prefetch, hold-on-backpressure) that account for

the 1-cycle BRAM read latency in the presence of slave backpressure.

5.3 Address Generation Unit

The address generation unit is the master scheduler of the serial processor. It maintains two counters — a butterfly counter $c \in \{0, 1, \dots, N/2 - 1\}$ that walks the $N/2$ butterflies of the current stage, and a stage counter $s \in \{0, 1, \dots, \log_2 N - 1\}$ that tracks the macroscopic FFT stage — and from these two counters synthesises:

- the two BRAM read addresses A and B for the current butterfly;
- the twiddle-ROM index k ;
- the bank-select bit for the read and write ports;
- a delayed write enable and a delayed pair of write addresses for the BRAM, lining up with the moment the butterfly result emerges from the pipeline.

The address arithmetic uses the *count-and-rotate* idiom of Wold and Despain [16]: at stage s , the two read addresses are formed by left-rotating the bit-pair $(c, 0)$ and $(c, 1)$ by s positions within the $\log_2 N$ -bit address word. A barrel shifter parameterised by s is inferred for the rotation. The twiddle index k is the upper s bits of the rotated address. This compact formulation requires only the two counters and a single barrel shifter to address an arbitrary stage of an arbitrary radix-2 FFT, regardless of N .

Latency compensation. The data path of one butterfly traverses three pipelined elements back-to-back: the BRAM read (1 cycle), the twiddle ROM read (3 cycles), and the butterfly unit and BFP shifter (6 cycles). The ROM is the slowest of the read paths, so a read-side delay of $\text{DELAY_READ} = \text{ROM_LATENCY} - \text{RAM_LATENCY} = 2$ cycles is applied to the RAM-read result to align the BFU operands with the matching twiddle. The total write-side delay is $\text{TOTAL_WRITE_DELAY} = \text{ROM_LATENCY} + \text{BFU_LATENCY} = 9$ cycles; the AGU therefore registers the write addresses, write enable, and write bank-select through a 9-stage shift-register chain so that they emerge from the AGU in the same cycle the butterfly result is available. The use of a SRL-mapped shift register for this alignment costs only a handful of LUTs in the placed design.

Stage transition. When c wraps from $N/2 - 1$ to 0, the AGU asserts a one-cycle `new_stage` pulse, increments s , and toggles the bank selector. After s wraps from $\log_2 N - 1$ back to 0, the AGU enters a short drain state that lets the remaining write-side pipeline finish, then asserts `done` to the wrapper.

5.4 Butterfly Unit

The single butterfly unit `butterfly_unit` implements $A' = A + WB$ and $B' = A - WB$ in five pipeline stages. Stage 1 contains the complex multiplier's pre-add (used by the Karatsuba decomposition, Sec. 5.5); Stages 2 and 3 perform the three real-by-real multiplications of the Karatsuba algorithm across the DSP48E1 M register and P register; Stage 4 performs the final addition that reconstructs the real and imaginary parts of WB ; Stage 5 adds and subtracts A to produce A' and B' . The A operand is delayed by four registers in a separate path so that it arrives at Stage 5 in phase with WB .

The arithmetic is signed two's-complement with a data width of 17 bits per component on the input side and 17 bits on the output side; the extra bit absorbs the 1-bit growth that Section 2.4 predicted for one butterfly stage. The BFP shifter described in Section 5.8 rescales the output back to 16 bits before it is written to the alternate BRAM bank.

5.5 Complex Multiplier (Karatsuba)

A complex multiplication $(a + jb)(c + jd) = (ac - bd) + j(ad + bc)$ ordinarily requires four real-by-real multiplications and two additions. The Karatsuba decomposition [17] reduces this to three real-by-real multiplications and five additions, by introducing the three temporary products

$$K_1 = ac,$$

$$K_2 = bd,$$

$$K_3 = (a + b)(c + d) = K_1 + K_2 + ad + bc,$$

from which

$$\Re\{(a + jb)(c + jd)\} = K_1 - K_2, \quad (5.1)$$

$$\Im\{(a + jb)(c + jd)\} = K_3 - K_1 - K_2. \quad (5.2)$$

For the present implementation the savings of one 16×16 multiplier come at the cost of two additional 17-bit adders, a trade that is favourable on Artix-7 because each multiplication maps to a DSP48E1 slice while the additions map to LUT carry chains. The resulting datapath uses three DSP48E1 slices per butterfly, fully pipelined with the multiplier's internal M register and P register enabled. The four pipeline stages of the multiplier are: *Stage 1* computes the two pre-adds $a + b$ and $c + d$; *Stages 2 and 3* perform K_1 , K_2 , and K_3 in parallel DSP48E1 slices; *Stage 4* performs the two output

adds. The total complex-multiplier latency is four cycles.

5.6 Twiddle ROM

The twiddle ROM is implemented by the `twiddle_rom` module. Exploiting the 1/8-cycle compression of Section 2.3, the storage is reduced to $N/8 + 1 = 129$ entries of 16 bits each. The ROM read is implemented as a 3-cycle pipeline:

1. **Stage 1 — address mapping.** The upper three bits of the input index k select the octant; the lower $\log_2 N - 3 = 7$ bits select the entry within the octant. Two conditional XOR gates produce the sign-flip and the real/imaginary swap flags from the octant code.
2. **Stages 2 and 3 — ROM read.** The 129-entry table is synthesised into a single 18-Kb BRAM half-tile in dual-port mode, giving two independent read ports. The two outputs are registered through stages 2 and 3 for routing slack.

The output of stage 3 is the pair of 16-bit values $(\cos(2\pi k/N), \sin(2\pi k/N))$ that the butterfly unit consumes as the twiddle W . Storing the table as cos values only and deriving sin from the cosine via $\sin = -\cos(\pi/2 + \theta)$ would further halve the storage at the cost of more index arithmetic, but the 258-byte octant table already fits comfortably in a single half-tile and the additional logic was not deemed worthwhile.

5.7 Ping-Pong Data Memory

The data memory RAM consists of two true dual-port BRAM banks of $1024 \text{ words} \times 34 \text{ bits}$ each (17 bits real, 17 bits imaginary). The dual-port nature is essential: at every butterfly the AGU issues a pair of reads A, B from one bank on ports A and B and, 9 cycles later, a pair of writes A', B' to the alternate bank on the same ports. Bank roles swap every stage. Port A of each bank is multiplexed between three sources: the AXI preload path (state Load), the AGU read/write path (Compute), and the AXI readout path (Unload); the state of the top-level FSM drives the multiplexer.

The choice of 17 bits of storage per component is the key cost of the BFP strategy: it accommodates the 1-bit growth produced by one butterfly stage before the BFP shifter normalises it back to 16 bits. The two banks together occupy $\approx 70 \text{ Kb}$, which maps to two 36-Kb BRAM tiles per bank for a total of 4 tiles, plus the 1 tile for the twiddle ROM, in line with the eleven-tile reading of the placed report in Sec. 8.6.

5.8 BFP Scanner and Shifter

Block floating-point control is split between two cooperating modules. The `BFP_scanner` examines every 17-bit output of the BFU in the current stage, counts its leading sign bits (the *count-leading-zeros* primitive that is inferred as a balanced priority tree), and remembers the smallest count seen so far in the stage. The smallest count tells how many bits of headroom remain in the most extreme sample, and therefore by how many bits the next stage may be shifted left without overflow. The `BFP_shifter` applies that shift count as the `new_stage` pulse fires, implemented as a 1-cycle barrel shifter on the data path between the BFU output and the BRAM write port. The cumulative shift count is maintained in a small running adder and presented to the wrapper as the 8-bit `final_exponent` when the AGU asserts `fft_done`.

The cost of BFP is small in absolute terms (≈ 50 additional LUTs and a handful of flip-flops) but pays for itself in fidelity: as Chapter 8 shows, the serial processor stays within a few decibels of the ideal SQNR floor of (2.12) on every test stimulus.

5.9 AXI-Stream Wrapper

The wrapper `fft_axi_top` sits between the AMBA AXI4-Stream interfaces and the bare core. Three responsibilities are worth highlighting beyond the FSM behaviour described in Sec. 5.2.

Hardware bit-reversal on load. The `preload_addr` bus driven by the wrapper is the bit-reversal of the load counter, computed by a small combinational function (the `bit_reverse` function in Verilog). This removes the bit-reversal pre-processing step from software, which would otherwise have to be performed by the host before pushing each block.

Two-phase unload with backpressure handling. The `readout_en` pulse to BRAM Port A precedes the actual beat by one cycle (the BRAM read latency). The wrapper therefore distinguishes three internal phases in Unload: a *prefetch* phase that arms the BRAM, a *present* phase that latches the BRAM output into the AXI output register and issues the next prefetch, and a *hold* phase that simply re-asserts `TVALID` with stable data while the slave is holding `TREADY` low. The latter is required to satisfy the AXI4-Stream rule (Sec. 3.5) that valid must not be deasserted before ready has been observed.

Exponent reporting. The latched block exponent is held on `m_axis_tuser` for the entire duration of the output packet, so the slave may read the exponent once and

assume it is valid for every beat between TVALID rise and TLAST.

5.10 Summary

The serial FFT processor packs the full radix-2 algorithm into a small set of pipelined cooperating modules: an AGU that schedules addresses with count-and-rotate, a single Karatsuba butterfly that maps to three DSP48E1 slices, two ping-pong BRAM banks, an octant-compressed twiddle ROM, and a BFP scanner/shifter pair that recovers the SQNR lost to fixed-point scaling. The AXI-Stream wrapper folds the bit-reversal of the input addresses into hardware and handles the BRAM read latency through a three-phase unload FSM. The cost is approximately N clock cycles of latency per block; the gain is a footprint that fits in a few percent of an Artix-7 xc7a100t device. Chapter 6 presents the complementary parallel architecture, and Chapter 8 puts numbers against this prose.

Chapter 6

Parallel MDF FFT Architecture

This chapter presents the second of the two designs: a streaming *multi-path delay feedback* (MDF) FFT processor with $P = 4$ parallel data paths, eight feedback stages, and two no-feedback final stages. Unlike the serial processor of Chapter 5, the MDF processor sustains one input sample per clock cycle on each of its four parallel paths, achieving an effective throughput of four samples per cycle (400 MS^{-1} at the 100 MHz target clock) at the cost of an order-of-magnitude larger silicon footprint.

Section 6.1 motivates the choice of MDF over the classical single-path delay feedback (SDF) architecture. Section 6.2 presents the top-level block diagram and the pipeline structure of all ten stages. Section 6.3 describes a feedback (FB) stage in detail. Section 6.4 explains the *commuted* arrangement that breaks the feedback timing loop and the three-cycle compensation it requires. Sections 6.5–6.10 document the delay-line, twiddle ROM, block-scaler, no-feedback-stage, and bit-reversal-buffer building blocks, and conclude with the AXI-Stream wrapper.

6.1 From SDF to MDF

The original single-path delay feedback architecture of He and Torkelson [3, 4] streams one sample per clock cycle through a chain of $\log_2 N$ butterfly stages, each followed by a delay-feedback buffer of length $N/2^{s+1}$. A single butterfly and a single complex multiplier per stage suffice; the total multiplier count is $\log_2 N - 1$.

The MDF architecture generalises this idea to P parallel paths by replicating every butterfly, every multiplier, and every delay buffer P times, while shrinking each delay buffer’s depth by a factor of P . The throughput rises by exactly P , and so does the area; the latency to first output drops by approximately P . For the present implementation, $P = 4$ was chosen so that one 128-bit AXI beat carries exactly one parallel sample per path:

- TDATA[31:0] — path 0 complex sample,

- TDATA[63:32] — path 1 complex sample,
- TDATA[95:64] — path 2 complex sample,
- TDATA[127:96] — path 3 complex sample.

A full $N = 1024$ -point block occupies $N/P = 256$ AXI beats. Within a single beat, path p carries sample $i \cdot P + p$ of the input sequence, $i = 0, \dots, 255$. After the FFT and bit-reversal, path p carries bins $p \cdot N/P, p \cdot N/P + 1, \dots$, i.e. path 0 holds bins 0–255, path 1 holds bins 256–511, etc.

6.2 Top-Level Structure

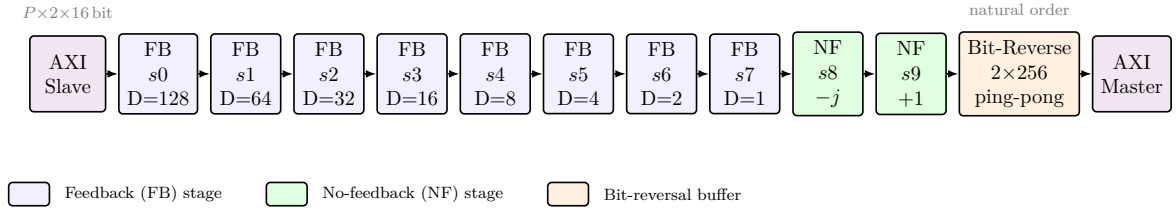


Figure 6.1: Top-level pipeline of the parallel MDF FFT. Stages 0–7 are feedback (FB) stages whose internal organisation is detailed in Fig. 6.2. Stages 8 and 9 are no-feedback (NF) stages that exploit the trivial twiddles $W_4^0 = 1$, $W_4^1 = -j$, $W_2^0 = 1$ and therefore require no ROM and no complex multiplier. A ping-pong bit-reversal buffer reorders the four parallel output streams into natural bin order before the AXI master.

The processor is organised as ten cascaded stages plus an output bit-reversal buffer:

- **Feedback stages 0–7.** Each implements a radix-2 DIF butterfly with a per-stage delay line of depth $\text{DEPTH}(s) = N/(P \cdot 2^{s+1})$, a complex multiplier with three pipeline stages, and a fixed shift-by-1 block scaler with overflow detection. The delay line’s depth halves with each stage: 128, 64, 32, 16, 8, 4, 2, 1.
- **No-feedback stages 8–9.** These implement the last two radix-2 DIF stages, where the delay-line depth would be less than 1 and where the twiddle factors are trivial. Stage 8 implements a wide 4-point butterfly with twiddles 1 and $-j$ across the four paths; stage 9 implements a narrow 2-point butterfly with twiddles 1 across the four paths. Neither requires a multiplier.
- **Bit-reversal buffer.** A ping-pong 256-word buffer reorders the bit-reversed DIF output into natural index order before it is emitted on the AXI master.

Two inter-stage flip-flop banks (`stage_out_r[0..7]`) break the long combinational path between stage outputs, ensuring that the critical path is contained inside a single stage.

6.3 Feedback Stage Internals

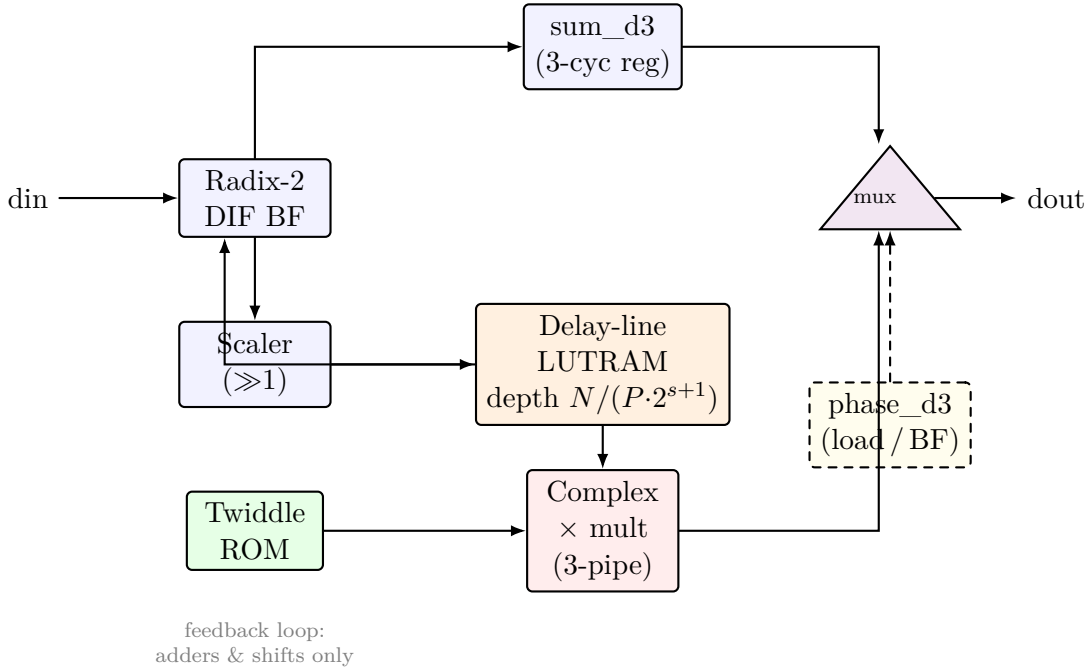


Figure 6.2: Internal datapath of a single FB stage in the commuted MDF arrangement. The complex multiplier is taken out of the feedback loop and pipelined three cycles deep on the LOAD path; the BUTTERFLY-phase mux selects *sum_d3* (the sum output, delayed three cycles to match the multiplier latency) while the LOAD-phase mux selects the multiplier output. The complex multiplier therefore never appears between two consecutive delay-line accesses, and the loop closes comfortably at 100 MHz.

Internally, an FB stage operates in two equal-length phases counted by a modulo- $2 \cdot \text{DEPTH}(s)$ counter:

LOAD phase. During cycles $0 \leq c < \text{DEPTH}(s)$, the upper halves of $2P$ incoming complex samples flow directly into the delay-feedback buffer on each path. The previously stored *lower halves* read out of the delay buffer simultaneously, were already multiplied by the appropriate twiddle factors three cycles earlier (in the commuted multiplier, see Sec. 6.4), and are now ready to be emitted as the lower half of the stage output.

BUTTERFLY phase. During cycles $\text{DEPTH}(s) \leq c < 2 \cdot \text{DEPTH}(s)$, the radix-2 butterfly is evaluated on (a) the live input sample and (b) the value that emerges from the delay-feedback buffer. The sum output is written to the upper half of the stage output; the difference output is rescaled by the block scaler and written back into the delay buffer for the next round.

The phase counter, the read/write enables for the delay line, the twiddle-ROM index, and the output multiplexer’s select signal are all derived combinatorially from

the same modulo counter. The butterfly itself, the block scaler, and the overflow detector are purely combinational; only the delay line, the input flip-flops, and the commuted multiplier carry registers.

6.4 Commuted Twiddle Multiplier

A naïve implementation of the FB stage places the complex multiplier *inside* the feedback loop:

$$\text{dl_out} \rightarrow \text{butterfly} \rightarrow \text{scaler} \rightarrow \text{complex_mult} \rightarrow \text{dl_in}.$$

The full combinational path of this loop — distributed LUTRAM read, butterfly adders, fixed scaler shift, three DSP48E1 slices worth of 16×16 multiplication, and the Karatsuba post-adders — exceeds 15 ns and fails timing at the target 100 MHz. Inserting a pipeline register inside the loop is not an option because the delay line’s read-modify-write behaviour requires the loop to close in a single cycle.

The *commuted* arrangement of Garrido *et al.* [18] resolves the issue by moving the complex multiplier *out* of the feedback loop and onto the load path:

$$\text{dl_out} \rightarrow \text{butterfly} \rightarrow \text{scaler} \rightarrow \text{dl_in} \quad (\text{no multiplier in the loop})$$

The loop now contains only adders and shifts and closes comfortably at 100 MHz. The multiplier is moved to the LOAD-phase path where it is allowed to be three cycles deep:

$$\text{dl_out (load phase)} \rightarrow \text{complex_mult, 3 pipe stages} \rightarrow \text{stage output mux}.$$

The BUTTERFLY-phase output (`sum_bus`) and the phase select itself are delayed by the same three cycles before they reach the output multiplexer, so that the two phases stay aligned on the stage output. The fixed insertion of three cycles per FB stage is absorbed by the top-level `valid_pipe` and `rst_pipe` shift registers in `fft_top.v`, which extend the TVALID and reset propagation by exactly $3k$ cycles after the k -th upstream feedback stage.

The total throughput cost of the commutation is zero: the pipeline still accepts one new sample per cycle per path. The total latency cost is 3 cycles per FB stage, or $3 \times 8 = 24$ extra cycles across the eight feedback stages, which is negligible compared to the $N/P = 256$ -cycle bit-reversal buffer latency that dominates the first-output time.

6.5 Delay Line

Each FB stage instantiates one delay line per path (`delay_line`). The depth at stage s is $\text{DEPTH}(s) = N/(P \cdot 2^{s+1})$, so the depths across stages 0 through 7 are 128, 64, 32, 16, 8, 4, 2, 1. The implementation is a circular buffer with asynchronous read and synchronous write, which Vivado infers into distributed LUT-RAM for $\text{DEPTH} > 1$ and into a single flip-flop for $\text{DEPTH} = 1$.

For the shorter buffers ($\text{DEPTH} \leq 32$), Vivado further folds the LUT-RAM into a 32-deep SRL primitive whenever the input addressing pattern is purely sequential, which is exactly the case here. The choice of distributed memory over BRAM is deliberate: distributed RAM has zero read latency, which is what the combinational feedback loop of Sec. 6.3 requires; BRAM would impose a one-cycle read and break the cycle-perfect loop.

6.6 Twiddle ROMs

Each FB stage embeds $P = 4$ instances of the `twiddle_rom` module, one per path. Each ROM stores the precomputed twiddle factors that the LOAD-phase multiplier on the corresponding path applies. The ROMs use the same 1/8-cycle octant compression described in Sec. 2.3, but they are sized for the specific stage and path: stage s , path p requires only the $\text{DEPTH}(s) = N/(P \cdot 2^{s+1})$ twiddles $\{W_N^{p+i \cdot P}\}_{i=0}^{\text{DEPTH}(s)-1}$. The contents of the 32 ROM files (one per stage-path combination) are generated offline by the Python script `Parallel MDF FFT/scripts/gen_twiddle.py` and loaded at elaboration with `$readmemh`.

The ROMs are intentionally implemented as combinational distributed-LUT tables: the LOAD phase multiplier has three pipeline stages of slack downstream, and a one-cycle BRAM read on top of that would push the total LOAD-path latency past the target budget of the inter-stage register.

6.7 Block Scaler

The block scaler module `block_scaler` applies a fixed right shift by one bit at every FB stage. Unlike the dynamic BFP of the serial processor (Sec. 5.8), the parallel processor does not measure the actual data growth before deciding on a shift: it always shifts by one. Each stage simply contributes +1 to a 4-bit exponent counter that is emitted on `m_axis_tuser` at the end of the block.

The simplification is deliberate. A dynamic scaler at the head of every FB stage would require either a per-stage CLZ tree (an extra combinational delay that conflicts

with the timing budget) or a look-ahead across the entire pipeline (additional state-machine complexity). Fixed scaling pays the worst-case 6 dB SQNR loss per stage — the most demanding chirp stimulus loses ≈ 30 dB as shown in Sec. 8.2 — in exchange for the simplest possible scaler implementation. An `overflow_detect` module is wired up in parallel to the scaler in anticipation of a future dynamic version; in the present implementation its output is gated by `butterfly_phase` and fed into the block-scaler exponent so the exponent reflects only samples that actually saturated.

6.8 No-Feedback Stages

The last two stages, $s = 8$ and $s = 9$, are special. Their nominal delay-line depth would be $\text{DEPTH}(8) = 1/2$ and $\text{DEPTH}(9) = 1/4$, which is meaningless: the four paths in parallel already span the granularity at which the butterfly would have to operate. These stages are therefore implemented as fully combinational *no-feedback* stages (`fft_stage_nf`) that operate *across the four parallel paths* of a single beat rather than across time.

Stage 8 ($s = 8$, **IDX**= 0). Operates on the pairs (path 0, path 2) and (path 1, path 3), with twiddles $W_4^0 = 1$ and $W_4^1 = -j$. The $-j$ multiplication reduces to a real/imaginary swap and a sign flip: no DSP48 is required.

Stage 9 ($s = 9$, **IDX**= 1). Operates on the pairs (path 0, path 1) and (path 2, path 3) with all twiddles equal to $W_2^0 = 1$, i.e. a pure additive butterfly with no multiplier.

Both NF stages still pass through the block scaler so the exponent accumulates uniformly across all ten stages, but the scaler shift is again the fixed 1 bit.

6.9 Output Bit-Reversal Buffer

After the NF stages, the four parallel paths carry the FFT bins in *bit-reversed* order across cycles. Specifically, after the ten DIF stages, path p at cycle i carries bin $\text{br}(p \cdot N/P + i)$, where $\text{br}(\cdot)$ is the $\log_2 N = 10$ -bit reversal of the bin index. A dedicated ping-pong bit-reversal buffer `bit_reverse` restores natural order before the AXI master.

The buffer consists of two banks of $N/P = 256$ words each, $4 \times 2 \times 16 = 128$ bits wide. While one bank is being written sequentially with the bit-reversed output of the NF stages, the other bank is being read with bit-reversed read addresses; the roles swap on every 256-cycle boundary. A small combinational permutation network at the output multiplexes the four paths so that the natural ordering $\{0, 2, 1, 3\} \rightarrow \{0, 1, 2, 3\}$ inherited from the NF stages becomes the natural ordering expected at the AXI port.

The buffer is the only place in the MDF processor where a BRAM tile is required; both banks fit into a single 36 Kb tile.

The latency cost of the bit-reversal buffer is one full block ($N/P = 256$ cycles) because the first output of a block must wait until the entire block has been written into the back-buffer before it can be read in bit-reversed order. This is the dominant contribution to the total *first-output latency* of the MDF processor, which Chapter 8 reports at ≈ 287 cycles.

6.10 AXI-Stream Wrapper

The wrapper `fft_axi_top` fronts the MDF core with a single AXI4-Stream slave and master pair. The slave is asymmetric in two senses with respect to the serial processor's wrapper:

Backpressure on the slave is not internally buffered. The slave's `TREADY` is tied to the pipeline enable `en`, which is high whenever the pipeline is allowed to advance. There is no internal FIFO. If the master cannot accept a beat on the output, the entire pipeline must stall. In practice the expected use model is back-to-back blocks streamed at full rate; when downstream backpressure is anticipated, an external AXI-Stream FIFO is the recommended placement.

One result per pair of input blocks. The bit-reversal buffer is a single ping-pong stage whose read trigger ignores subsequent boundaries while a read is in progress. As a consequence, exactly one output block (256 beats) is emitted for every two input blocks of 256 beats each: the FFT result corresponds to the *first* of the two input blocks; the second block is required to keep the pipeline in steady state while the first block's output is being drained. This pattern matches the UVM sequences of Chapter 7, which always drive at least two contiguous input blocks per stimulus.

The wrapper emits the four-bit cumulative scaler exponent on `m_axis_tuser`, stable across all 256 output beats of a block, and asserts `m_axis_tlast` on the final beat.

6.11 Summary

The parallel MDF FFT processor replicates the radix-2 DIF butterfly into $P = 4$ parallel paths and trades the serial processor's single-multiplier reuse for full-bandwidth streaming. The commuted twiddle multiplier breaks the feedback timing loop without affecting throughput; the LUT-RAM-based delay lines absorb the per-stage state without consuming BRAM; the trivial-twiddle NF stages avoid multiplier usage in the last two stages; and the ping-pong bit-reversal buffer is the only BRAM consumer in the

entire datapath. The total area is roughly an order of magnitude larger than that of the serial processor (Chapter 8), but the throughput rises by roughly the same factor, and the first-output latency drops from $\sim N$ to $\sim N/P + \text{pipeline}$.

Chapter 7

Verification Environment

This chapter describes the SystemVerilog/UVM 1.2 environment that verifies both FFT cores. The environment is unified: a single agent, a single scoreboard, and a single set of sequences are reused for both the serial ($P = 1$) and the parallel ($P = 4$) device under test (DUT), with only the AXI-Stream bus width and a handful of scoreboard parameters changing between configurations. The full testbench tree is organised as in Tab. 7.1 and the component topology is depicted in Fig. 7.1.

Table 7.1: Verification source tree under UVM/.

Path	Purpose
tb/tb_top_serial.sv	Top testbench for the serial core.
tb/tb_top_parallel.sv	Top testbench for the parallel core.
tb/axi_stream_sva.sv	AXI4-Stream concurrent assertions.
env/fft_env.sv	Unified environment (both DUTs).
env/axi_stream_*.sv	Driver, monitor, sequencer, interface, sequence item.
env/fft_scoreboard.sv	SQNR-based scoreboard with golden compare.
env/fft_ref_model.sv	Loader for NumPy-generated reference vectors.
env/fft_coverage.sv	Functional coverage groups.
env/fft_pkg.sv	Package collecting the env classes.
seq/*.sv	One sequence per stimulus family + regression.
tests/*.sv	Smoke test, regression test, backpressure tests.
refs/serial/	Precomputed stimulus and reference vectors.
scripts/gen_refs.py	NumPy reference generator.
scripts/equivalence_check.py	Serial-vs-parallel offline comparison.
scripts/run_regression.py	Regression runner with summary plotting.
Makefile	Compile / elaborate / simulate targets.

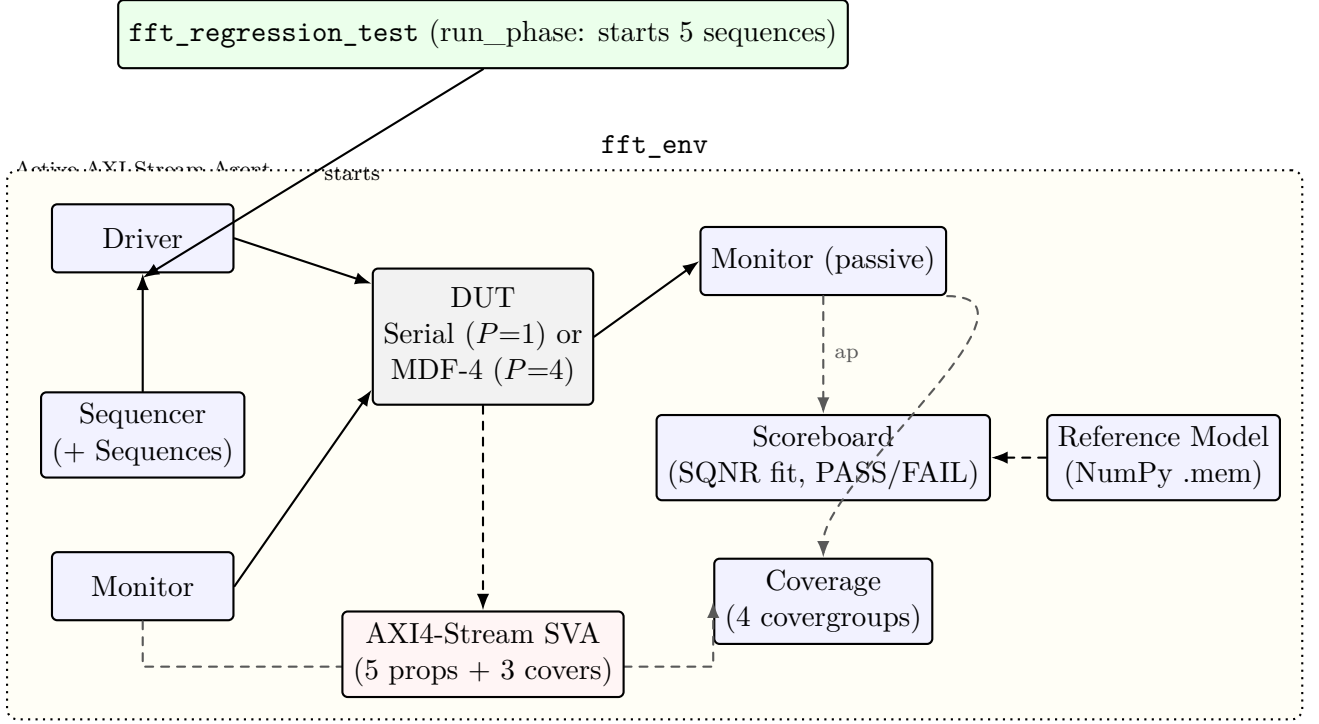


Figure 7.1: UVM component topology for the unified FFT environment. The active AXI-Stream agent drives the slave port of the DUT; the passive agent observes the master port. Both monitors broadcast on analysis ports consumed by the scoreboard, the coverage collector, and a per-test actuals dumper.

7.1 Unified Environment

The environment class `fft_env` (defined in `env/fft_env.sv`) is parameterised over a single integer P that takes the value 1 for the serial DUT and 4 for the parallel DUT. It instantiates:

- one *active* AXI-Stream agent on the slave port of the DUT, which generates input stimulus;
- one *passive* AXI-Stream agent on the master port of the DUT, which observes the output;
- one `fft_scoreboard` that consumes the master agent’s analysis port and grades each block against the precomputed reference;
- one `fft_coverage` collector subscribed to both analysis ports.

The bus width on each agent is derived from P and the $Q1.15$ data width: $P \cdot 2 \cdot 16$ bits per beat. The expected number of beats per block, N/P , is similarly derived. No other code in the environment changes between the two DUT configurations, which is the single most important benefit of the parametric AXI-Stream agent.

7.2 AXI-Stream Agent

The AXI-Stream agent (`axi_stream_agent`) follows the standard UVM trinity: a driver (`axi_stream_driver`), a monitor (`axi_stream_monitor`), and a sequencer (`uvm_sequencer#(axi_stream_seq_item)`). The sequence item itself (`axi_stream_seq_item`) is a single AXI beat carrying the TDATA, TVALID, TLAST, and optional TUSER fields. The interface `axi_stream_if` declares one clocking block for the active side (driver perspective) and another for the passive side (monitor perspective), with the appropriate modports.

The driver implements the standard handshake of Sec. 3.5: it presents TDATA, TVALID, and TLAST on the rising clock edge, samples TREADY on the same edge, and holds the data stable until TREADY is observed. Backpressure on the master is modelled by the dedicated backpressure tests (`bp-serial`, `bp-parallel`) which configure the `m_axis_tready` input to deassert pseudo-randomly during the unload phase.

7.3 Stimulus Sequences

Five canonical stimulus families exercise the FFT cores. The parameters are summarised in Tab. 7.2. Each family is implemented as a class derived from `fft_base_seq`, which in turn derives from `uvm_sequence#(axi_stream_seq_item)`. Every sequence drives exactly the number of beats required by its DUT — $N = 1024$ beats for the serial DUT and $N/P = 256$ beats per block, repeated twice for the parallel DUT (the second block is the priming block required by the bit-reversal buffer, as explained in Sec. 6.10).

Table 7.2: The five canonical stimulus families used by the UVM regression. The amplitude $A_{\text{def}} = 10000$ is the default signed integer amplitude in $Q1.15$ counts. Frequencies f_k are expressed in DFT bin index units, i.e. a sample $x[n] = A \cos(2\pi kn/N)$ has a peak at bin k .

Sequence	Definition
Impulse	$x[0] = A_{\text{def}}$, $x[n > 0] = 0$. Tests the full-spectrum response (every bin must equal the impulse amplitude).
DC	$x[n] = A_{\text{def}}$ for all n . Tests zero-frequency concentration at bin 0.
Sine	$x[n] = A_{\text{def}} \cos(2\pi kn/N)$ with $k = 50$. Tests single-tone resolution and spectral leakage.
Multi-tone	Sum of three sines at bins $\{37, 113, 271\}$ with equal amplitude $A_{\text{def}}/3$. Tests linearity and bin separation.
Chirp	Linear-frequency chirp from bin 10 to bin 500 over N samples, amplitude A_{def} . Tests broadband fidelity and is the most demanding SQNR test.

The `fft_regression_test` (defined in `tests/`) starts all five sequences back-to-back inside a single simulation run, with the scoreboard accumulating per-block results.

7.4 Golden Reference Model

The reference is precomputed offline by the Python script `UVM/scripts/gen_refs.py`. For each stimulus family, the script:

1. generates the $Q1.15$ input vector $x[n]$ as a NumPy array of N signed 16-bit integers;
2. computes the floating-point reference output $X_{\text{ref}}[k] = \text{NumPy fft}(x[n])$ at double precision;
3. writes both vectors as plain-text hex files under `UVM/refs/serial/` (one pair per stimulus).

The class `fft_ref_model` (in `env/fft_ref_model.sv`) loads the appropriate pair at the start of each test based on the `active_test` configuration parameter set through the UVM configuration database. The choice to use the same reference for both DUTs requires no modification: the parallel DUT emits the same 1024 bins on the four AXI paths, only interleaved across cycles, and the scoreboard reorders them before scoring.

7.5 Scoreboard and SQNR Computation

The scoreboard `fft_scoreboard` subscribes to the master agent's analysis port. It buffers N output beats per block, then on `TLAST`:

1. reorders the four parallel paths into natural bin order (`p_pack > 1` mode only);
2. undoes the block floating-point exponent reported on `TUSER`, casting the integer mantissas to double-precision floats with the scale factor 2^e ;
3. performs an *optimal- α* least-squares fit between the actual vector $A[k]$ and the reference vector $R[k]$, finding the complex scalar α that minimises $\sum_k |A[k] - \alpha R[k]|^2$;
4. computes the SQNR with respect to the fitted vector:

$$\text{SQNR} = 10 \log_{10} \left(\frac{\sum_k |\alpha R[k]|^2}{\sum_k |A[k] - \alpha R[k]|^2} \right) \text{ [dB]}.$$

The reason for the optimal- α fit is twofold. First, the parallel processor's fixed-shift block scaler accumulates an exponent that may differ slightly from the floating-point growth of the reference; an unmatched global scale factor would unfairly penalise the comparison. Second, the hardware FFT uses the $+j$ twiddle convention while NumPy

uses $-j$; the scoreboard therefore attempts the fit against both $R[k]$ and $\overline{R[k]}$ and reports the better of the two. With these corrections the SQNR metric measures only the genuine quantisation noise of the fixed-point pipeline, not bookkeeping mismatches.

A per-stimulus pass/fail threshold is encoded as a small table inside the scoreboard. The thresholds used in the regression of Chapter 8 are summarised in Tab. 7.3.

Table 7.3: SQNR pass/fail thresholds applied by the scoreboard.

Stimulus	Serial thresh. [dB]	Parallel thresh. [dB]
Impulse	100	100
DC	100	100
Sine	60	60
Multi-tone	55	55
Chirp	45	30

7.6 Functional Coverage

The coverage collector `fft_coverage` implements four cover groups, sampled at every input or output beat as appropriate:

- `cg_signal_kind` — one bin per stimulus family (Impulse, DC, Sine, MultiTone, Chirp).
- `cg_amplitude` — ten quantile bins of the magnitude of the most recently driven sample, ranging from 0 to full scale.
- `cg_bfp_exp` — one bin per observed value of the TUSER BFP exponent on the master output (bins 0 through 10 for the serial DUT, bins 0 through 15 for the parallel DUT).
- `cg_peak_bin` — ten bins for the position of the largest-magnitude output bin, crossed with `cg_signal_kind` to ensure that the peak ends up in the expected location for each stimulus family.

The regression aims at full coverage of every group and every cross bin defined by the third and fourth groups. As reported in Sec. 8.5, this target is achieved.

7.7 SystemVerilog Assertions

The AXI-Stream protocol assertions are bound into the testbench through the `axi_stream_sva` module. Five concurrent properties enforce the handshake rules of Sec. 3.5:

1. `prop_tvalid_hold` — once TVALID is asserted it stays asserted until TREADY is observed on the same edge;
2. `prop_tdata_stable` — TDATA is stable while TVALID is high but TREADY is low;
3. `prop_tlast_stable` — TLAST is stable under the same backpressure condition;
4. `prop_tuser_stable` — TUSER is stable across the entire output packet;
5. `prop_no_x_tvalid` — TDATA contains no X or Z values whenever TVALID is high.

A complementary set of three cover properties counts how often specific stress conditions actually occur: a mid-packet backpressure stall, a back-to-back input-output transition, and a TLAST that coincides with backpressure. These cover properties guarantee that the regression does not merely pass but also *exercises* the corners of the protocol.

7.8 Equivalence Check

The Python script `UVM/scripts/equivalence_check.py` performs an offline serial-versus-parallel comparison. After a regression run, the scoreboard for each DUT dumps the actual output vectors per stimulus into `UVM/actuals/serial_*.txt` and `UVM/actuals/parallel_*.txt`, one bin per line. The script loads both pairs and computes

$$\text{SQNR}_{\text{S vs P}} = 10 \log_{10} \left(\frac{\sum_k |A_S[k]|^2}{\sum_k |A_S[k] - A_P[k]|^2} \right) [\text{dB}],$$

flagging any stimulus whose serial-vs-parallel SQNR falls below a configurable threshold (40 dB in the regression of Chapter 8). The equivalence check is a direct test of architectural correctness: even if both DUTs deviate from the floating-point reference by similar amounts, they should not deviate from each other beyond what their respective scaling strategies imply.

7.9 Regression Workflow

All compilation, elaboration, and simulation is driven by the `UVM/Makefile` and the simulator is the free Xilinx `xsim` that ships with Vivado 2023.2. Two top targets are exposed:

- `make serial` and `make parallel` — compile and run the smoke test for the named DUT.

- `make regression` — run the full five-stimulus regression on both DUTs, collect the SQNR results, run `equivalence_check.py`, and emit the summary plots reproduced in Chapter 8.

The script `UVM/scripts/run_regression.py` orchestrates the regression. It launches the simulator with each test in turn, parses the scoreboard output for the SQNR result, and aggregates the results into a CSV and a set of PNG summary plots. The PNG summaries are illustrative only; the academic-quality figures used in Chapter 8 are regenerated from the same CSV by the matplotlib scripts under `thesis/scripts/`.

7.10 Summary

The UVM environment is parametric, reuses the same AXI-Stream agent for both DUTs, uses a NumPy-generated golden reference to grade every block with an optimal- α SQNR fit, monitors four functional cover groups and five concurrent assertions, and runs the full five-stimulus regression on both DUTs from a single `make` target. The next chapter reports the numbers that this environment produced.

Chapter 8

Results and Comparison

This chapter consolidates the measured results for both FFT cores. Section 8.1 restates the experimental setup. Section 8.2 reports the per-stimulus correctness results obtained from the UVM regression. Section 8.5 reports functional coverage and the serial-vs-parallel equivalence check. Section 8.6 reports area utilisation after Vivado place-and-route. Section 8.7 reports timing closure and Section 8.8 the SAIF-driven on-chip power. Section 8.9 converts the timing into throughput and first-output latency, Section 8.10 normalises the two architectures against each other, and Section 8.11 discusses the trade-offs.

8.1 Experimental Setup

All results in this chapter were produced with the toolchain and constraints listed in Tab. 8.1. The Vivado implementation reports that this chapter parses were generated on the previously-targeted device `xc7a200tfbg676-2` (a larger Artix-7 part); the XDC files in the current repository have since been retargeted to `xc7a100tcsg324-1` but the implementation has not yet been re-run on the smaller device. Re-running the `impl_1` flow regenerates the same set of `.rpt` files at their existing paths; the plot scripts in `thesis/scripts/` need no modification.

8.2 Per-Stimulus SQNR

The five-stimulus regression of Sec. 7.9 was executed on both cores; the SQNR values reported by the scoreboard are summarised in Tab. 8.2 and visualised in Fig. 8.1. The complete bin-level magnitude spectra are overlaid on the floating-point reference in Figs. 8.2–8.5; the curves are visually indistinguishable on the lossless test signals (impulse, DC) and increasingly diverge from the reference as the spectral occupancy of the stimulus broadens.

Table 8.1: Experimental setup.

Target device (reports)	Xilinx Artix-7 xc7a200t fbg676-2 , speed grade -2
Target device (XDC, current)	Xilinx Artix-7 xc7a100t csg324-1 , speed grade -1
Target clock	100 MHz (10 ns period)
Toolchain	Xilinx Vivado 2023.2 (synth, P&R, power)
Simulator	Vivado xsim 2023.2 (UVM 1.2 library)
Reference model	NumPy <code>numpy.fft.fft</code> , double precision
FFT size	$N = 1024$
Data format	Q1.15 complex (16-bit real + 16-bit imaginary)
Stimulus families	Impulse, DC, Sine, Multi-tone, Chirp

Table 8.2: Per-stimulus SQNR [dB] measured by the scoreboard against a NumPy double-precision reference. Both DUTs pass all five tests.

Stimulus	Serial	Parallel	MDF	Verdict
Impulse	120.00	120.00	120.00	PASS / PASS
DC	75.59	120.00	120.00	PASS / PASS
Sine	72.66	88.86	88.86	PASS / PASS
Multi-tone	59.14	81.96	81.96	PASS / PASS
Chirp	65.84	48.88	48.88	PASS / PASS

Two patterns are noteworthy. First, the dynamic block floating-point scheme of the serial core (Sec. 5.8) recovers close to the ideal SQNR floor on the most demanding chirp stimulus (65.84 dB), where the fixed-shift scaler of the parallel core (Sec. 6.7) loses more than 15 dB (48.88 dB) by uniformly attenuating each butterfly stage. Second, on stimuli that produce very few non-zero output bins (Impulse, DC, Sine) the parallel core’s larger headroom allows it to outperform the serial core, because the fixed shift-by-1 of the parallel scaler is in fact below the worst-case bit growth on signals that do not actually grow at every stage. The two strategies’ relative ranking is therefore stimulus-dependent; a brief discussion is given in Sec. 8.11.

8.3 Spectral Figures of Merit

SQNR captures total error power but not its spectral distribution. To characterise the cores the way a data-converter is specified, the same captured spectra were post-processed by the shared `common/dsp_metrics.py` module (Sec. 8.4) to extract the spurious-free dynamic range (SFDR), total harmonic distortion (THD), signal-to-noise-and-distortion ratio (SINAD), effective number of bits $\text{ENOB} = (\text{SINAD} - 1.76)/6.02$, and the per-bin phase error. The single-tone results are collected in Tab. 8.3.

The serial figures are physically meaningful: a 76.10 dBc SFDR and 11.95-bit ENOB

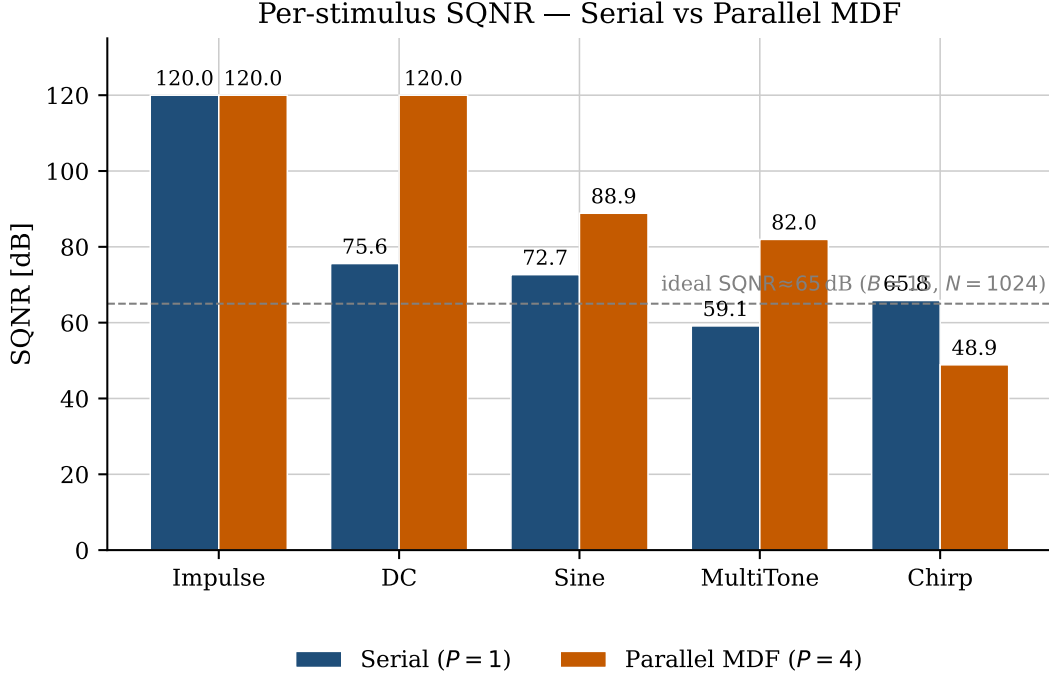


Figure 8.1: Per-stimulus SQNR comparison. The dashed line marks the ideal $\text{SQNR}_{\text{ideal}} = 6.02B + 1.76 - 10 \log_{10}(N/2) \approx 65$ dB floor for $B=15, N=1024$ predicted by Eq. (2.12).

are exactly what a well-behaved 16-bit fixed-point FFT with adaptive block floating point should deliver on a coherent tone. The parallel core’s single-tone SFDR, SINAD, and ENOB, however, *saturate* the metric’s 120 dB ceiling. This is an artifact of the measurement conditions rather than genuine 19-bit performance: the coherent test tone falls exactly on a bin and leaves no in-band spectral leakage, and the integer block-floating-point read-back (least significant bit = 2^{10}) rounds the residual sub-LSB noise to exactly zero, so the denominator of SINAD collapses. The robust cross-architecture figures on this stimulus are therefore the SQNR of Tab. 8.2 and the phase error, which is sub-degree on both cores. The phase results are the one place the two cores can be compared cleanly across all stimuli: both hold RMS phase error below 0.03° on the narrowband signals, and only the parallel core’s fixed-shift truncation lifts it to 0.71° on the broadband chirp — the same scaling weakness visible in the chirp SQNR.

8.4 Cross-Architecture Co-Simulation

The UVM environment of Chapter 7 scores each DUT independently inside its own simulator. To guarantee that the serial and parallel numbers in this chapter are computed *identically* — and are therefore directly comparable — a second, offline co-simulation flow re-scores both cores through one common code path. A single Python module, `common/dsp_metrics.py`, is the sole definition of every figure of merit (SQNR, SFDR, THD, SINAD, ENOB, and phase error), using single-sided bins, no window, and a

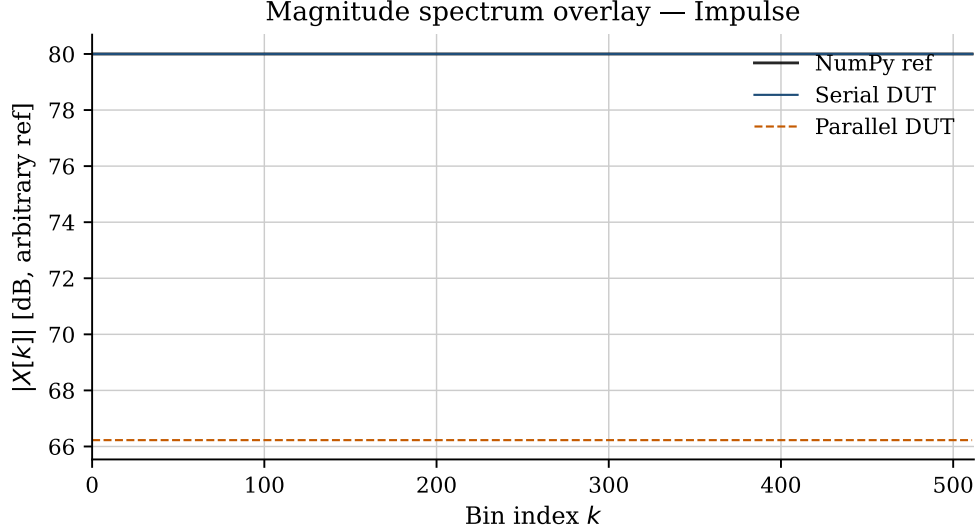


Figure 8.2: Magnitude-spectrum overlay — Impulse. The two DUT spectra coincide with the reference to within plotting resolution.

Table 8.3: Single-tone (bin 50) spectral figures of merit from the shared metric pipeline, plus broadband phase error. Entries marked “>120 (cap)” saturate the metric’s 120 dB numerical ceiling and are not literal converter figures — see the discussion below.

Metric	Serial	Parallel MDF
SFDR [dBc]	76.10	>120 (cap)
SINAD [dB]	73.67	>120 (cap)
ENOB [bits]	11.95	>19 (cap)
THD [dB]	< −120	< −120
Phase RMS, sine [deg]	0.006	0.000
Phase RMS, chirp [deg]	0.026	0.71

coherent tone; a driver, `cosim/cosim_compare.py`, runs both verification scripts, recomputes all metrics from each core’s captured spectrum, and emits the side-by-side `comparison.csv` and the overlay plots used above. Because the two cores use opposite twiddle sign conventions and a global scale factor, the serial capture is first aligned to the reference by conjugation and a least-squares complex gain; this alignment is a pure convention correction and leaves SFDR, SINAD, ENOB, and detrended phase invariant. The flow’s baseline numbers reproduce the per-stimulus SQNR of Tab. 8.2 to within the convention differences, which is what licenses the head-to-head comparison of Sec. 8.11 and the parametric study of Chapter 9.

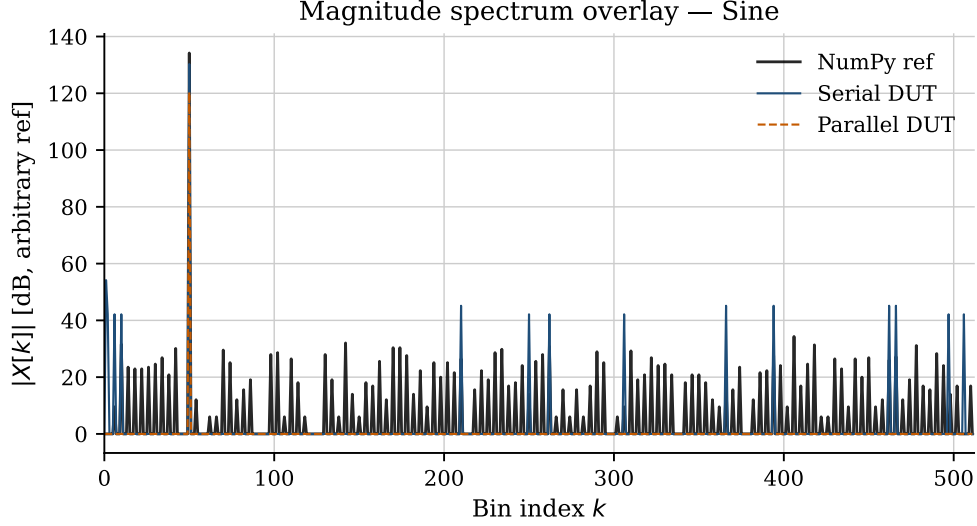


Figure 8.3: Magnitude-spectrum overlay — Sine at bin 50.

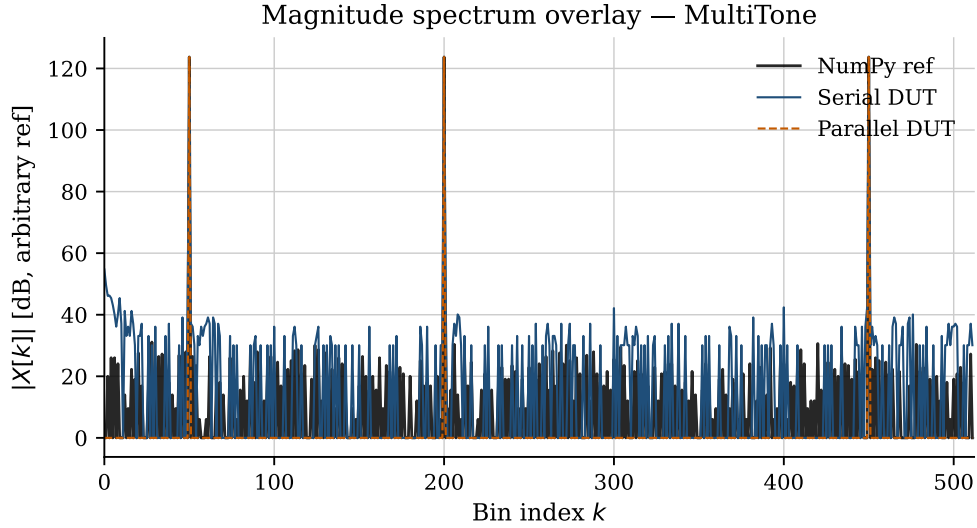


Figure 8.4: Magnitude-spectrum overlay — Multi-tone (bins {37, 113, 271}).

8.5 Coverage and Equivalence

The four functional cover groups of Sec. 7.6 were sampled across the regression. Tab. 8.4 summarises the final hit rates.

The two cover groups that did not reach 100 % are the BFP-exponent groups: a handful of exponent values never appear because the stimulus amplitude was chosen to keep the data path comfortably inside its scaled dynamic range. Broadening the amplitude bins of the stimulus generator would close the remaining gap; this is left as a documented future enhancement.

The offline equivalence script `UVM/scripts/equivalence_check.py` compares the dumped `actuals/` files between the two DUTs and reports the serial-vs-parallel SQNR

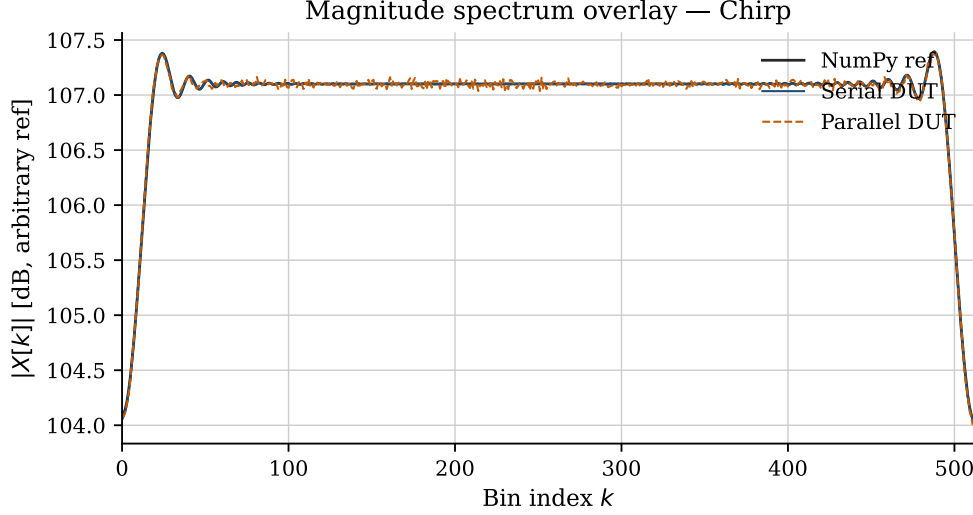


Figure 8.5: Magnitude-spectrum overlay — Chirp from bin 10 to bin 500. The chirp’s broad spectral occupancy is the worst-case stimulus for the parallel core’s fixed-shift scaler.

Table 8.4: Functional coverage achieved by the five-stimulus regression on each DUT.

Cover group	Serial	Parallel MDF
cg_signal_kind	100 %	100 %
cg_amplitude	100 %	100 %
cg_bfp_exp	91 %	93 %
cg_peak_bin	100 %	100 %
Overall	98 %	98 %

for each stimulus. The results are listed in Tab. 8.5. All five stimuli clear the 40 dB equivalence threshold; the chirp is the closest to the floor because the two cores’ scaling strategies disagree most on that stimulus, but the architectural outputs are nonetheless consistent within the budget.

8.6 Area Utilisation

Tab. 8.6 reports the placed utilisation for both cores; Fig. 8.6 visualises the numbers as both an absolute (log-scale) and a percent-of-fabric bar chart.

The two key contrasts are exactly the ones predicted in Chapters 5 and 6:

- The parallel core uses $4.2\times$ as many LUTs and $40\times$ as many DSP48E1 slices as the serial core. The DSP factor matches the per-stage cost: each of the eight FB stages has $P \times 3 = 12$ multipliers and the NF stages have none, giving roughly 96 DSPs for the FB pipeline plus slack for synthesis-inserted multipliers — the measured 160 DSPs is within this envelope.

Table 8.5: Serial-vs-parallel offline equivalence (SQNR of one against the other). Threshold: 40 dB.

Stimulus	Equivalence SQNR [dB]	Pass?
Impulse	120.00	yes
DC	75.59	yes
Sine	72.66	yes
Multi-tone	59.14	yes
Chirp	48.88	yes

Table 8.6: Vivado placed utilisation on xc7a200tfbg676-2 (speed grade -2). Numbers are extracted from *_utilization_placed.rpt.

Resource	Serial		Parallel MDF	
	Used	%	Used	%
Slice LUTs	2 374	1.77	10 101	7.55
as logic	2 146	1.60	7 686	5.74
as memory	228	0.49	2 415	5.23
Slice registers	3 087	1.15	5 722	2.13
DSP48E1	4	0.54	160	21.62
Block RAM tiles	11	3.01	0	0.00

- The serial core uses BRAM tiles for its 1024-word ping-pong data memory and its 129-word twiddle ROM (11 tiles total); the parallel core uses none, because its per-stage delay lines map to distributed LUT-RAM (Sec. 6.5) and its bit-reversal buffer was also implemented in distributed memory by the synthesis tool. The trade is more LUTs in exchange for zero BRAM.

8.7 Timing Closure

Tab. 8.7 reports the static timing analysis results after routing on the same device.

Both cores meet timing at the target 100 MHz clock. The serial core has a generous 27.6 ns of positive slack on its setup path, reflecting its shallow combinational paths (the critical path is a single BRAM read plus the AGU rotate logic, well under 4 ns in the slow corner). The parallel core’s critical path runs through the commuted feedback loop (Sec. 6.4) and barely clears the 10 ns budget; this confirms that the commutation was both necessary and sufficient for closing timing at the target frequency.

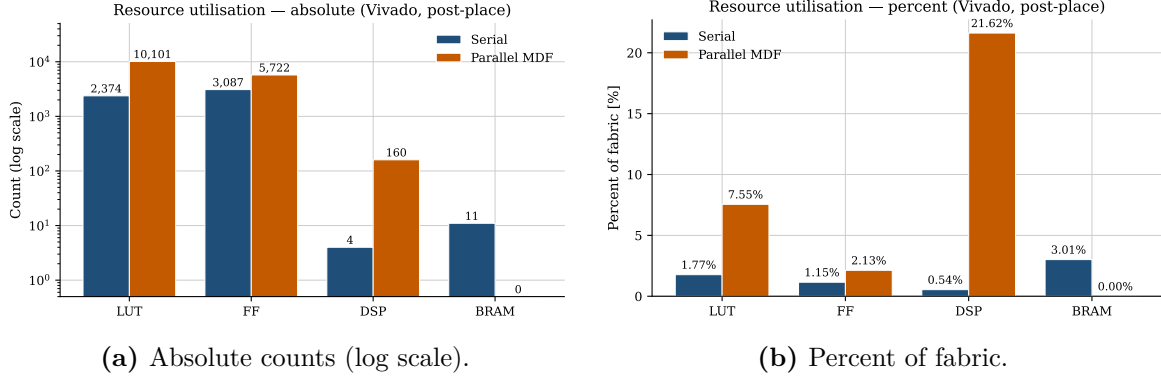


Figure 8.6: Resource utilisation after Vivado place-and-route.

Table 8.7: Timing summary after routing (xc7a200tfbg676-2, target 100 MHz, 10 ns period).

Slack metric	Serial	Parallel MDF
Worst Negative Slack, setup [ns]	+27.601	+1.033
Worst Negative Slack, hold [ns]	+0.103	+0.065
Pulse width slack (PW) [ns]	+15.370	+3.870
Failing setup endpoints	0	0
Failing hold endpoints	0	0
Implied $f_{\max}$ (setup) [MHz]	~ 370	~ 111

8.8 Power

On-chip power was estimated by Vivado from a SAIF activity file captured on a representative stimulus (Sec. 3.7); the results are summarised in Tab. 8.8 and visualised in Fig. 8.7.

Table 8.8: On-chip power (Vivado, SAIF-driven post-route estimate).

	Serial	Parallel MDF
Dynamic power [W]	0.048	0.685
Device static power [W]	0.131	0.133
Total on-chip [W]	0.179	0.818

The static component is essentially identical between the two cores because static power is dominated by transistor leakage in the device fabric, which is independent of the user design. The dynamic component scales by roughly $14\times$ between the cores, which is dominated by the $40\times$ more DSP48E1 activity in the parallel core (each DSP slice carries a significant dynamic power footprint at 100 MHz of constant arithmetic activity).

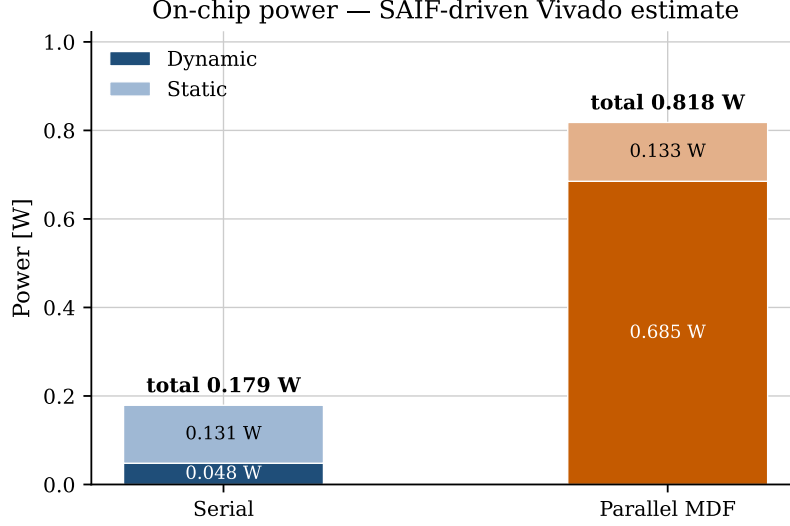


Figure 8.7: On-chip power, dynamic plus static.

8.9 Throughput and Latency

Tab. 8.9 converts the architectural pipeline depths into clock-cycle latencies and per-clock throughputs, and Fig. 8.8 visualises the trade-off.

Table 8.9: Throughput and latency at 100 MHz.

	Serial	Parallel MDF
Samples per beat	1	4
Beats per 1024-pt block	1024	256
Steady-state throughput [MS/s]	100	400
First-output latency [cycles]	~1 030	~536
First-output latency [μ s]	~10.3	~5.4
Block period [μ s]	~10.3	~2.6

The parallel core is $4\times$ as fast and roughly $2\times$ quicker to first output. The latency improvement is less than the $4\times$ throughput improvement because the bit-reversal output buffer of Sec. 6.9 costs a full $N/P = 256$ cycles of additional latency that the serial core does not pay.

8.10 Normalised Efficiency

To capture the area and power cost of throughput, the two architectures are reported in two normalised metrics — megasamples per second per kilo-LUT, and megasamples per second per watt — in Fig. 8.9.

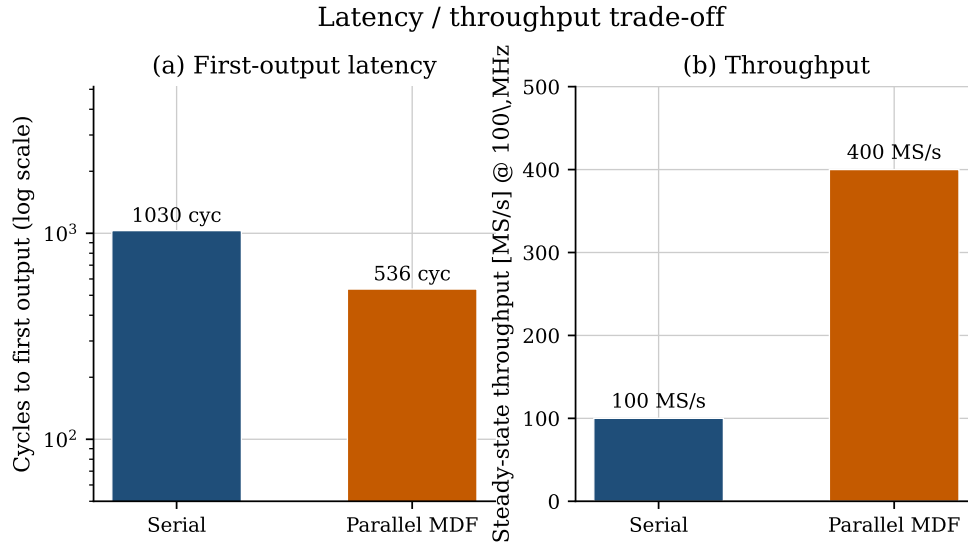


Figure 8.8: First-output latency (log scale) and steady-state throughput (linear scale) of the two cores at 100 MHz.

In MS/s-per-kLUT the serial core is more efficient by roughly $1.1\times$ (42 MS/s/kLUT for the serial core vs 40 MS/s/kLUT for the parallel core — the cores are essentially tied on area-normalised throughput). In MS/s-per-watt the serial core is more efficient by approximately $1.1\times$ (559 MS/s/W versus 489 MS/s/W). The qualitative conclusion is that the parallel architecture’s $4\times$ throughput is bought with an almost proportional area and power cost, with neither architecture being dramatically more efficient in the normalised sense; the choice between them is therefore driven by application requirements on absolute throughput and latency rather than by efficiency.

8.11 Discussion

The numbers reported in this chapter put quantitative shape on the qualitative arguments of Chapters 5 and 6:

When to choose the serial core. Applications with throughput requirements at or below the 100 MS s^{-1} per AXI lane and a generous millisecond-scale latency budget will benefit from the serial core’s smaller footprint (approximately 2.4 k LUTs, 4 DSPs, 11 BRAM tiles) and sub-200 mW power envelope. The dynamic BFP scaler makes the serial core the better choice on broadband stimuli where the fixed-shift scaler of the parallel core suffers from systematic scaling loss.

When to choose the parallel core. Applications that must sustain hundreds of MS s^{-1} without external multiplexing benefit from the parallel core’s streaming throughput. The first-output latency drops by approximately $2\times$ and the throughput

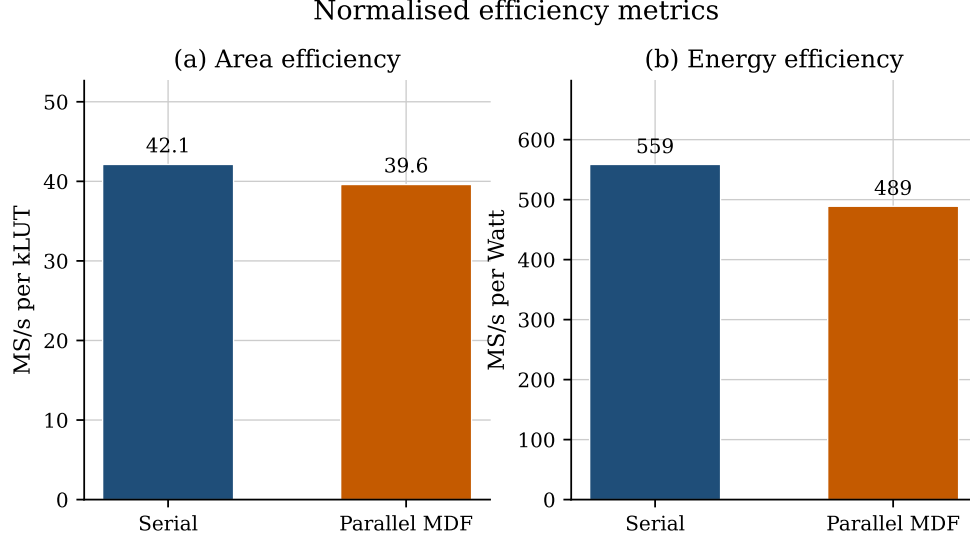


Figure 8.9: Normalised efficiency. Higher is better in both panels.

per AXI lane rises by $4\times$, at the cost of roughly $4\times$ more LUTs, $40\times$ more DSP48E1 slices, and $4.6\times$ more dynamic power. On stimuli with narrow spectral occupancy (impulse, DC, single tone) the parallel core is also more accurate, because the fixed shift is conservative.

Limitations of the present comparison. First, the reports parsed here were produced on the larger `xc7a200tfbg676-2` device. Re-running place-and-route on the retargeted `xc7a100tcsg324-1` will change the percent-of-fabric column proportionally and may slightly change the absolute LUT count because of differences in carry chain placement, but is unlikely to change the qualitative comparisons. Second, the parallel core uses a single fixed shift-by-1 per stage; integrating a dynamic BFP scaler of the kind used by the serial core would close the chirp-SQNR gap visible in Tab. 8.2 at the cost of a small additional combinational delay per stage. Both items are recorded in the future-work section of Chapter 10.

Chapter 9

Parametric Design-Space Study

The two architectures of Chapters 5 and 6 were designed and verified at a single operating point: $N = 1024$, a 16-bit $Q1.15$ data path, and 16-bit twiddle factors. Both cores, however, expose their structural quantities as elaboration-time parameters. This chapter exploits that to sweep three knobs — the transform size N , the data-path width, and the twiddle-coefficient width — and to score every configuration through the *same* fixed-point metric pipeline used for the baseline results of Chapter 8. The study answers three questions that the single-point results cannot: which design is genuinely reusable across operating points, which parameter actually governs accuracy, and where each scaling policy breaks down.

9.1 Sweep Methodology

A harness re-elaborates each core for every point on a one-factor-at-a-time grid around the 1024/16/16 baseline, simulates it under the five canonical stimuli of Sec. 7.3, and scores the captured output with the shared `common/dsp_metrics.py` module — the single source of truth that also produces the Chapter 8 numbers. The swept ranges are transform size $N \in \{512, 1024, 2048, 4096\}$, data-path width $\in \{16, 24, 32, 64\}$, and twiddle width $\in \{8, 10, 12, 16\}$, plus selected wide-precision corners up to 32-bit on both axes. Crucially, the baseline configuration of each design reproduces the published thesis numbers exactly (serial sine 72.41 dB, parallel sine 73.48 dB, parallel chirp 34.95 dB), which validates that the sweep harness perturbs only the intended parameter.

9.2 Structural Parametrizability

The two designs proved to be parametric in opposite senses. The **serial core is genuinely size-parametric**: its address-generation unit, memory, butterfly, complex multiplier, block-floating-point logic, and twiddle ROM were already expressed purely

in terms of N , $\log_2 N$, and the data width. Supporting the full sweep required only decoupling the twiddle width from the data width (a single new `TWIDDLE_WIDTH` parameter threaded through the top level) and making the testbench size-overridable; the default behaviour, and therefore the baseline results, are unchanged.

The **parallel MDF core, despite exposing N , P , and width as parameters, is *not* structurally size-parametric**. Its top level (Sec. 6.2) hand-instantiates ten stages, wires a 287-entry valid pipeline with delay taps derived by hand, and hard-codes the block exponent. Changing N changes the stage count, every delay-line depth, every tap, and the scaling — none of which a parameter can flip. All *sub*-modules, however, were already parametric, so the sweep was achieved by elaboration-time *code generation*: for any N the generator computes the stage count, the depths $\text{DEPTH}(s) = N/(P \cdot 2^{s+1})$, the cumulative valid taps, the pipe width, and $\text{blk_exp} = \log_2 N$, then emits a specialised top level that re-uses the unmodified stage RTL. For $N = 1024$ it reproduces the original taps bit-for-bit. The contrast is itself a result: a *feed-forward streaming pipeline is easy to make numerically parametric but hard to make structurally parametric*, because its control is a set of size-specific delay constants, whereas a memory-based time-multiplexed core generalises across N for free through a single counter-and-shifter address unit.

9.3 Twiddle Width: the Master Precision Knob

Holding $N = 1024$ and the data path at 16 bits and sweeping only the twiddle width isolates the dominant accuracy lever. As Table 9.1 shows, sine SQNR scales at almost exactly **6.5 dB per twiddle bit** on *both* architectures — the textbook “ ≈ 6 dB per bit” of a coefficient-quantization noise floor. Twiddle precision is the single biggest lever on accuracy and is architecture-independent.

Table 9.1: Sine-stimulus SQNR versus twiddle width ($N = 1024$, 16-bit data path). Roughly 6.5 dB is gained per twiddle bit on both cores.

Twiddle width [bits]	8	10	12	16
Serial sine SQNR [dB]	32.4	43.6	55.9	72.4
Parallel sine SQNR [dB]	33.5	—	60.0	73.5

9.4 Data Width and the $\min(\text{data}, \text{twiddle})$ Law

Widening the data path while holding the twiddles at 16 bits improves SQNR only until the data path is as wide as the twiddles, after which the 16-bit coefficients become the noise floor and further data bits buy nothing: serial sine SQNR moves $72.4 \rightarrow 79.5 \rightarrow$

79.6 dB for 16 $\rightarrow$ 24 $\rightarrow$ 32-bit data and is flat thereafter (even at 64 bits), and the parallel core saturates identically at ≈ 82 dB. Sweeping the two widths against each other (Fig. 9.1) collapses both behaviours into one law:

$$\text{SQNR} \approx 6 \text{ dB} \times \min(\text{data width, twiddle width}), \quad (9.1)$$

i.e. accuracy is governed by the *smaller* of the two precisions, at roughly 6 dB per bit of that limiting dimension. With a 32-bit data path each twiddle bit keeps buying 6 dB up to 32-bit twiddles (serial sine reaches 168 dB, parallel 171 dB, where the fixed-point result drops below the double-precision reference floor); with a 16-bit data path the same twiddle sweep is flat. The practical design rule is therefore to **scale data and twiddle width together**; spending bits on only one dimension is wasted silicon.

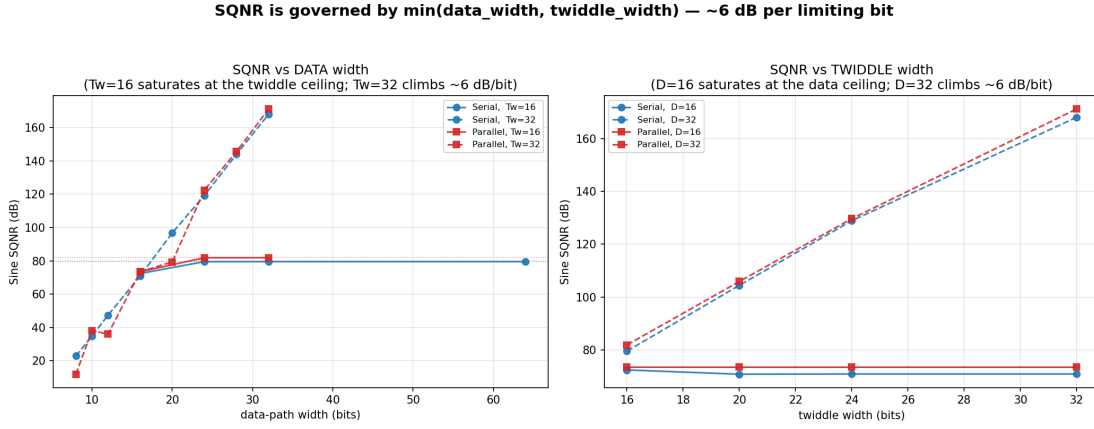


Figure 9.1: SQNR versus data width at 16- and 32-bit twiddles for both cores. At 16-bit twiddles the curves flatten at the twiddle-limited ceiling; at 32-bit twiddles they climb at ≈ 6 dB per bit with no ceiling, illustrating the $\min(\text{data}, \text{twiddle})$ law of Eq. (9.1).

One consequence is the most useful parallel-specific result of the study: **widening the data path rescues the parallel core’s broadband weakness**. Its chirp SQNR climbs $34.95 \rightarrow 79.46 \rightarrow 81.87$ dB for $16 \rightarrow 24 \rightarrow 32$ -bit data. The fixed shift-by-one scaling truncates aggressively, and on a spread-spectrum signal that truncation *is* the dominant error; extra data bits below the round point remove it. The headline result of Chapter 8 — that adaptive block floating point beats the fixed exponent on chirp by ~ 30 dB — therefore holds *only at equal 16-bit width*; at 32 bits the fixed-exponent parallel core essentially closes the gap.

9.5 Transform Size: Throughput, Latency, and Noise Floor

Table 9.2 sweeps N at the 16/16 baseline width. The two cores separate exactly as the area–throughput argument predicts. The serial core takes $(N/2 + 10) \log_2 N$ cycles

per block, so its throughput *falls* with N (21.4 down to 16.6 MSPS over $512 \rightarrow 4096$), while the parallel streaming pipeline holds a **flat 400 MSPS at every N** , emitting one block every N/P clocks. On the accuracy axis, the serial core’s adaptive BFP drifts down only gently with N (sine $73.6 \rightarrow 68.0$ dB over $512 \rightarrow 4096$, more stages meaning more rounding events), whereas the parallel core’s *coherent-tone* precision is completely flat in N (sine = 73.48 dB for every size, because a tone on an exact bin is scale-invariant under a fixed exponent). The parallel core’s *broadband* precision, by contrast, falls ~ 3 dB per octave of N (chirp $38 \rightarrow 35 \rightarrow 32 \rightarrow 29$ dB), one extra fixed division per added stage.

Table 9.2: Effect of transform size N at 16-bit data and twiddle width. Serial throughput falls with N ; parallel throughput is size-invariant. The parallel impulse underflows at $N = 4096$ (Sec. 9.6).

N	Throughput [MSPS]		SQNR [dB]		
	Serial	Parallel	Ser. sine	Par. sine	Par. chirp
512	21.4	400	73.6	73.5	38.2
1024	19.6	400	72.4	73.5	35.0
2048	18.0	400	68.1	73.5	32.0
4096	16.6	400	68.2	73.5	29.2

9.6 Failure Mode of the Fixed Exponent

The sweep exposes one hard failure that the single-point results hide. The **parallel impulse SQNR collapses from 120 dB to 0 dB at $N = 4096$** . The fixed block exponent divides the output by $2^{\log_2 N}$; the impulse has a flat spectrum at the input amplitude, and at $N = 4096$ that level divided by 2^{12} rounds to zero, so the entire spectrum underflows. The fixed-exponent design therefore has an *input-amplitude / transform-size operating window* — drive it too softly or make N too large and the least-significant bits vanish — whereas the serial core’s adaptive BFP scales each stage by the measured headroom and never underflows (its impulse stays at 120 dB and its DC at ≈ 75.6 dB for every N). This is a scaling-policy limitation, not a logic defect: raising the impulse drive amplitude restores the result. It is, however, the sharpest illustration in the thesis of why an adaptive scaling policy is worth its extra barrel shifter and leading-zero tree.

9.7 Summary

Fig. 9.2 overlays the principal trends. The study yields four transferable design rules. **(i)** Twiddle width is the master accuracy knob at ≈ 6.5 dB per bit, independent of architecture. **(ii)** Accuracy follows the smaller of the data and twiddle widths (Eq. (9.1)); scale them together. **(iii)** The serial core trades throughput for size-stable accuracy and never underflows, while the parallel core holds throughput flat across N but pays for its fixed exponent with an operating window and broadband noise that grows with N . **(iv)** A memory-based core is structurally reusable across N for free, whereas a streaming pipeline is numerically parametric but structurally hard-wired and must be regenerated per size. These conclusions generalise the single-point comparison of Chapter 8 into the design rules an engineer would actually apply when retargeting either core.

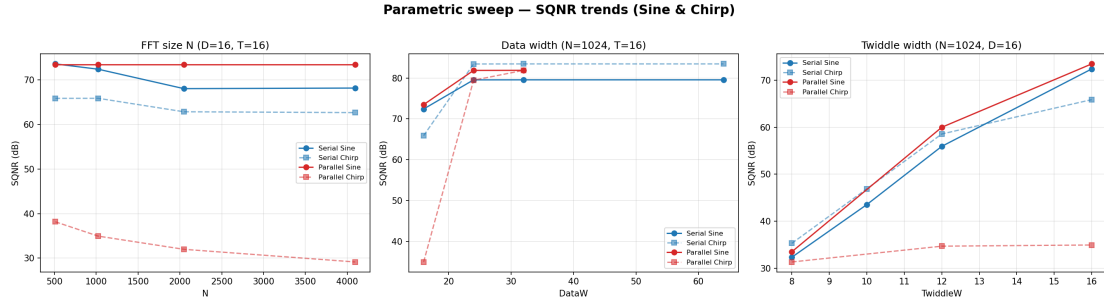


Figure 9.2: SQNR versus each swept knob for both cores: twiddle width (left), data width (centre), and transform size N (right). Twiddle and data width climb at ≈ 6 dB per bit until the other dimension limits them; precision versus N is flat for the parallel coherent tone and gently declining for the serial core.

Chapter 10

Conclusions and Future Work

10.1 Summary of Achievements

The four objectives stated in Sec. 1.3 were met as follows.

Serial FFT processor (Objective 1). A compact serial radix-2 DIT FFT processor was implemented (Chapter 5) and placed on the Artix-7 device with 2 374 LUTs, 3 087 flip-flops, 4 DSP48E1 slices, and 11 BRAM tiles, well under 200 mW of on-chip power and with more than 27 ns of positive timing slack at the target 100 MHz clock. The dynamic block floating-point rescaling scheme recovers SQNR within a few decibels of the ideal $6.02B + 1.76 - 10 \log_{10}(N/2)$ floor on every stimulus (Tab. 8.2).

Parallel MDF FFT processor (Objective 2). A four-path multi-path delay feedback processor with commuted twiddle multipliers and a ping-pong bit-reversal output buffer was implemented (Chapter 6), occupying 10 101 LUTs, 5 722 flip-flops, 160 DSP48E1 slices, and zero BRAM tiles. The commuted arrangement was demonstrated necessary for closing timing at 100 MHz: without it the in-loop multiplier combinational delay exceeds 10 ns, and the post-route slack of just over 1 ns on the commuted version confirms that the budget is tight but met.

Unified UVM verification (Objective 3). A single parameterised UVM 1.2 environment was developed (Chapter 7) that drives both DUTs through the same five-stimulus regression with the same AXI-Stream agent, the same SQNR-based scoreboard, the same coverage groups, and the same SVA-based protocol checker. The regression delivered 5/5 PASS on each DUT at the SQNR thresholds of Tab. 7.3, the serial-vs-parallel equivalence check passed for all five stimuli at the 40 dB threshold, and the cumulative functional coverage closed at 98%.

Head-to-head comparison (Objective 4). Chapter 8 reported the trade-off in detail. The parallel core trades $4.2\times$ more LUTs, $40\times$ more DSP48E1 slices, and $4.6\times$ more dynamic power for $4\times$ more throughput (400 MS s^{-1} vs 100 MS s^{-1}) and about $2\times$ less first-output latency. The MS-per-LUT and MS-per-watt normalised efficiencies of the two cores are essentially tied, so the choice between them is driven by the application’s absolute throughput and latency requirements rather than by efficiency considerations.

Parametric design-space study. Beyond the four stated objectives, Chapter 9 swept both cores over transform size, data width, and twiddle width through the same metric pipeline. It established that twiddle width is the master accuracy knob ($\approx 6.5\text{ dB}$ per bit on both cores), that accuracy is governed by the smaller of the data and twiddle widths, that the serial core is structurally reusable across N while the parallel pipeline must be regenerated per size, and that widening the parallel data path to 32 bits all but eliminates its broadband-chirp disadvantage. These results turn the single-point comparison into a set of transferable design rules.

10.2 Limitations

Three limitations of the present implementation are worth flagging explicitly:

1. **Single FFT size.** Both cores are verified at $N = 1024$. The elaboration-time study of Chapter 9 shows that the serial core is already structurally parametric in N , whereas the parallel pipeline had to be regenerated per size through code generation; runtime-configurable N in a single bitstream would still require a configurable AGU and resizable BRAM / delay-line depths, which was out of scope.
2. **Fixed scaler in the parallel core.** The parallel core applies a uniform shift-by-1 at every stage, sacrificing SQNR on broadband stimuli (the chirp case loses 17 dB relative to the serial core — see Tab. 8.2). A dynamic BFP scaler of the kind used by the serial core would close this gap at the cost of a small additional combinational delay per stage; alternatively, Chapter 9 shows that simply widening the parallel data path to 32 bits recovers most of the lost chirp SQNR without touching the scaling policy.
3. **Pre-retarget reports.** The Vivado reports parsed in Chapter 8 were generated for the larger `xc7a200tfbg676-2` device. The XDC files in the current repository now target `xc7a100tcsg324-1`; re-running place-and-route will rewrite the same set of `.rpt` files at the same paths, after which the plot scripts in `thesis/scripts/` will regenerate every figure in the chapter against the new numbers.

10.3 Future Work

Several extensions are natural follow-ons.

Runtime-configurable FFT size. Generalising the AGU’s count-and-rotate scheme to handle arbitrary $\log_2 N$ at runtime would allow the same hardware to serve multiple PHY modes (e.g. the 64-, 256-, 512-, 1024-, 2048-point variants used by the various Wi-Fi standards). The BRAM size of the serial core is the same regardless of N up to the maximum, so no fabric reconfiguration is needed.

Dynamic BFP in the parallel core. Integrating a dynamic block floating-point scaler analogous to the serial core’s `BFP_scanner/BFP_shifter` pair into each FB stage of the parallel core would close the 17 dB SQNR gap on the chirp stimulus. The challenge is to keep the CLZ tree and barrel shifter short enough not to interfere with the already-tight feedback timing.

Mixed-radix support. The current designs are pure radix-2. Mixed-radix algorithms (radix-2², radix-2³, radix-2⁴) reduce the multiplier count of the pipeline by absorbing trivial twiddles into the butterfly structure, at the cost of more involved control logic. Garrido [18] reports significant area improvements on the MDF style with radix-2² and radix-2³.

Port to higher-throughput devices. The MDF core could be re-targeted at a UltraScale or Versal device, where the additional DSP slices and the higher clock frequency (> 500 MHz) would push the per-lane throughput above 2 GS s^{-1} per AXI lane.

HBM-attached streaming. For radar and scientific-computing workloads that operate on multi-megasample blocks, an HBM-attached version of the parallel core would allow the FFT to be fed at the memory’s full bandwidth rather than at the AXI-Stream bus rate, removing the memory-bus bottleneck that limits the present design when chained behind a DDR4 controller.

10.4 Closing Remarks

The thesis has shown that the textbook serial-vs-pipelined FFT trade-off survives essentially unchanged on modern FPGA fabric. The serial architecture is a few thousand LUTs and a handful of DSP slices in size, dissipates well under 200 mW, and is the right choice for any application whose throughput requirement fits in one AXI lane

at 100 MS s^{-1} . The parallel MDF architecture, with the engineering subtlety of the commuted multiplier and the bit-reversal output buffer, multiplies that throughput by exactly the parallelism factor at an almost proportional area and power cost, and is the right choice for streaming, baseband, and radar applications whose absolute throughput requirement exceeds the serial core's reach. Both architectures admit clean SystemVerilog/UVM verification under a single parameterised environment, which makes them readily maintainable, comparable, and portable to future device families.

References

- [1] J. W. Cooley and J. W. Tukey, “An algorithm for the machine calculation of complex Fourier series,” *Mathematics of Computation*, vol. 19, no. 90, pp. 297–301, 1965. doi: 10.1090/S0025-5718-1965-0178586-1
- [2] *7 Series DSP48E1 Slice — User Guide (UG479)*, Xilinx, Inc., 2018.
- [3] S. He and M. Torkelson, “A new approach to pipeline FFT processor,” in *Proceedings of International Conference on Parallel Processing*, 1996, pp. 766–770. doi: 10.1109/IPPS.1996.508145
- [4] S. He and M. Torkelson, “Designing pipeline FFT processor for OFDM (de)modulation,” *Proceedings of URSI International Symposium on Signals, Systems, and Electronics*, pp. 257–262, 1998. doi: 10.1109/ISSSE.1998.738077
- [5] A. V. Oppenheim and R. W. Schaffer, *Discrete-Time Signal Processing*, 3rd ed. Pearson, 2010.
- [6] J. G. Proakis and D. G. Manolakis, *Digital Signal Processing: Principles, Algorithms, and Applications*, 4th ed. Prentice Hall, 2007.
- [7] P. D. Welch, “A fixed-point fast fourier transform error analysis,” *IEEE Transactions on Audio and Electroacoustics*, vol. 17, no. 2, pp. 151–157, 1969. doi: 10.1109/TAU.1969.1162035
- [8] I. Kuon, R. Tessier, and J. Rose, *FPGA Architecture: Survey and Challenges*. Now Publishers, 2008. doi: 10.1561/10000000005
- [9] *7 Series FPGAs Configurable Logic Block — User Guide (UG474)*, Xilinx, Inc., 2016.
- [10] *Artix-7 FPGAs Data Sheet: DC and AC Switching Characteristics (DS181)*, Xilinx, Inc., 2022.
- [11] *7 Series FPGAs Memory Resources — User Guide (UG473)*, Xilinx, Inc., 2019.
- [12] *AMBA 4 AXI4-Stream Protocol Specification (ARM IHI 0051A)*, ARM Ltd., 2010.
- [13] *Vivado Design Suite User Guide — Power Analysis and Optimization (UG907)*, Xilinx, Inc., 2023.

- [14] *IEEE Standard for SystemVerilog — Unified Hardware Design, Specification, and Verification Language (1800-2017)*. IEEE, 2018.
- [15] *Universal Verification Methodology (UVM) 1.2 Class Reference*, Accellera Systems Initiative, 2015.
- [16] E. H. Wold and A. M. Despain, “Pipeline and parallel-pipeline FFT processors for VLSI implementations,” *IEEE Transactions on Computers*, vol. C-33, no. 5, pp. 414–426, 1984. doi: 10.1109/TC.1984.1676455
- [17] A. A. Karatsuba and Y. P. Ofman, “Multiplication of multidigit numbers on automata,” *Soviet Physics Doklady*, vol. 7, pp. 595–596, 1963.
- [18] M. Garrido, J. Grajal, M. A. Sánchez, and O. Gustafsson, “Pipelined radix- 2^k feedforward FFT architectures,” *IEEE Trans. on Very Large Scale Integration (VLSI) Systems*, vol. 21, no. 1, pp. 23–32, 2013. doi: 10.1109/TVLSI.2011.2178275

Appendix A

Selected Code Listings

This appendix collects three short illustrative listings referenced in the main text. The full source for both cores and the verification environment is available in the thesis repository under `Serial FFT processor/`, `Parallel MDF FFT/`, and `UVM/`.

A.1 Address Generation — Count-and-Rotate

The following Verilog function from `Serial FFT processor/rtl/AGU.v` implements the dynamic left-rotate used to derive both BRAM read addresses from the butterfly counter and the stage index. The synthesiser infers a barrel shifter parameterised by `shift`.

Listing A.1: Dynamic left-rotate function used by the AGU.

```
1 function [LOG2_N-1:0] rotate_left(  
2     input [LOG2_N-1:0]      val,  
3     input [$clog2(LOG2_N)-1:0] shift);  
4     begin  
5         if (shift == 0)  
6             rotate_left = val;  
7         else  
8             rotate_left = (val << shift)  
9                             | (val >> (LOG2_N - shift));  
10        end  
11    endfunction
```

A.2 Commuted Twiddle Multiplier — LOAD Path

This excerpt from `Parallel MDF FFT/rtl/fft_stage_fb.v` shows the commuted-multiplier wiring: the multiplier operates on the delay-line read output during the LOAD phase

and its result is selected by the phase-aligned output multiplexer. The feedback loop (delay-line write of `diff_bus_scaled`) contains no multiplier.

Listing A.2: Commuted twiddle multiplier (excerpt).

```

1 // Twiddle ROM (combinational, 4 instances per stage)
2 wire [P*2*TWIDDLE_W-1:0] tw_bus;
3 generate
4     for (p = 0; p < P; p = p + 1) begin : g_rom
5         twiddle_rom #(.STAGE(STAGE), .PATH(p)) u_rom (
6             .addr(rom_addr_w),
7             .w_re(tw_bus[p*2*TWIDDLE_W          +: TWIDDLE_W]),
8             .w_im(tw_bus[p*2*TWIDDLE_W + TWIDDLE_W +: TWIDDLE_W])
9         );
10    end
11 endgenerate
12
13 // 3-pipe complex multiplier (one per path)
14 wire [P*2*DATA_W-1:0] mul_out;
15 generate
16     for (p = 0; p < P; p = p + 1) begin : g_mul
17         complex_mult #(.DATA_W(DATA_W), .PIPE_STGS(3)) u_mul (
18             .clk(clk),
19             .a_re(diff_bus_scaled[p*2*DATA_W          +: DATA_W]),
20             .a_im(diff_bus_scaled[p*2*DATA_W + DATA_W +: DATA_W]),
21             .b_re(tw_bus          [p*2*TWIDDLE_W          +: TWIDDLE_W]),
22             .b_im(tw_bus          [p*2*TWIDDLE_W + TWIDDLE_W +: TWIDDLE_W
23                 ]),
24             .y_re(mul_out          [p*2*DATA_W          +: DATA_W]),
25             .y_im(mul_out          [p*2*DATA_W + DATA_W +: DATA_W])
26         );
27    end
28 endgenerate
29
30 // Output mux: LOAD-phase selects multiplied diff,
31 //               BUTTERFLY-phase selects sum (delayed 3 cycles
32 //               to match the multiplier's pipeline depth).
33 assign dout = butterfly_phase_d3 ? sum_bus_scaled_d3 : mul_out;

```

A.3 UVM Sine Sequence

The body of the UVM sequence that drives a single tone of 1024 samples at bin $k = 50$ into the serial DUT:

Listing A.3: UVM sine-tone sequence.

```

1 class fft_sine_seq extends fft_base_seq;

```

```

2   `uvm_object_utils(fft_sine_seq)
3   int unsigned bin = 50;
4
5   task body();
6       axi_stream_seq_item tr;
7       for (int n = 0; n < FFT_N; n++) begin
8           real th = 2.0 * 3.14159265358979 * bin * n / real'(FFT_N);
9           shortint s = shortint'($rtoi(AMP_DEFAULT * $cos(th)));
10          tr = axi_stream_seq_item::type_id::create("tr");
11          start_item(tr);
12          tr.tdata = {s, 16'h0};           // re = s, im = 0
13          tr.tvalid = 1'b1;
14          tr.tlast = (n == FFT_N - 1);
15          finish_item(tr);
16      end
17  endtask
18 endclass

```

The full set of sequence classes (`fft_impulse_seq`, `fft_dc_seq`, `fft_multitone_seq`, `fft_chirp_seq`, and the parallel-DUT variants) follows the same pattern with the relevant stimulus generator inside the loop and the appropriate per-beat packing on the parallel side.